STUDY MATERIAL

Foundation for AI and Data Science

- **15 MARKS IN Data Science [DTSC] ---- CLASS XI**

Syllabus for Section 3 : Foundation for AI and Data science [15 Marks] - ARTI

- **History of AI : Alan Turing and cracking enigma , mark 1 machines , 1956- the birth of the term AI , AI winter of 70's , expert systems of 1980s, skipped journey of present day AI, pattern recognition and Machine learning**

- **Brief History of Data Science , Data Science as a conjunction of computer science statistics and domain knowledge . Definition of Data Science , Data Science Life Cycle – capture , maintain , process , analyze , communicate**

- **Introduction to Linear Algebra and statistics for AI :**
  Basic matrix operations like matrix addition , subtraction , multiplication , transpose of matrix , identity matrix.
  Brief introduction to vectors , unit vector , normal vector , Euclidean space.
  Probability distribution , frequency , mean , median and mode , variance and standard deviation , Gaussian distribution.
  Correlation , Regression , parametric , non-parametric tests (Basic idea)
  Distance function , Euclidean norm , distance between two points in 2D and 3D and extension of idea to n dimensions.

- **Basic ideas of different Data Science Toolkit : Excel , R**

- **CONTENTS**

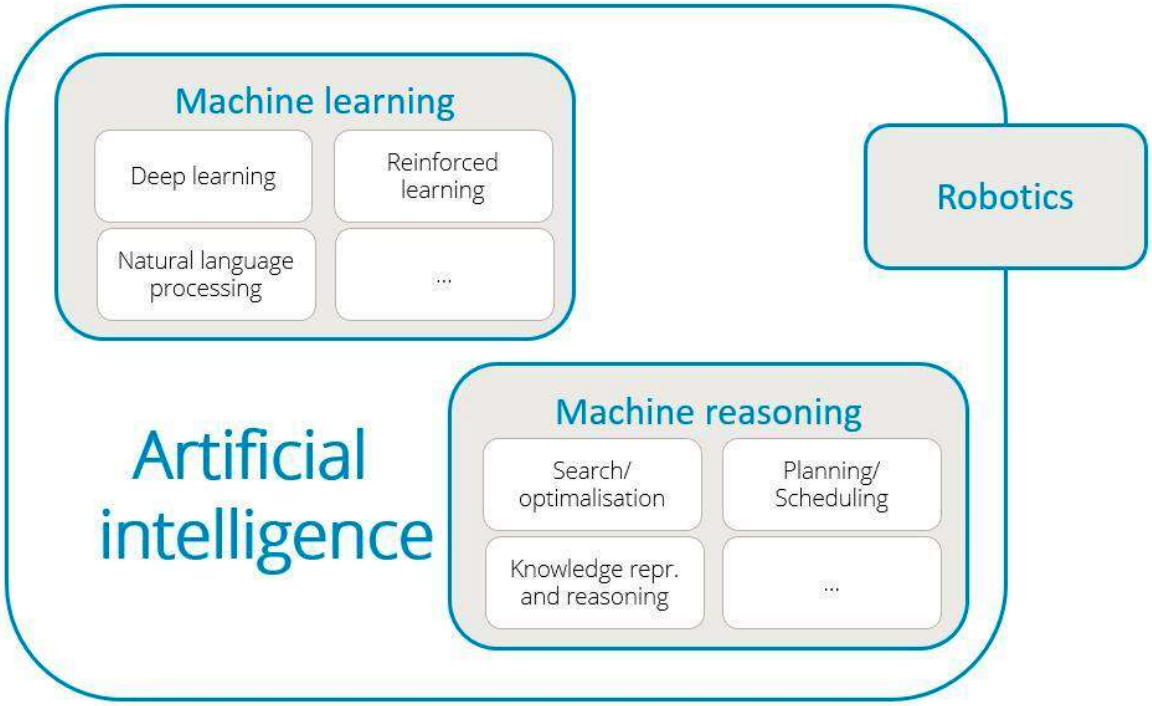
Sl No	Topic	Page No
1	AI-Basic Concept	4
2	History of AI	18
3	Turing Test	28
4	Data Science – Basic Concept	31
5	Distance function , Euclidean Norm , Distance between two points in 2D , 3D and n dimension	35
6	Correlation and Regression	39
7	Uniform Distribution , parametric , non-parametric tests	52
8	Data Science Tool Kit	64
9	R	69

What Is Artificial Intelligence?

Artificial Intelligence is currently one of the hottest buzzwords in tech and with good reason. The last few years have seen several innovations and advancements that have previously been solely in the realm of science fiction slowly transform into reality.

Experts regard artificial intelligence as a factor of production, which has the potential to introduce new sources of growth and change the way work is done across industries. China and the United States are primed to benefit the most from the coming AI boom, accounting for nearly 70% of the global impact.

Artificial Intelligence is a method of making a computer, a computer-controlled robot, or a software think intelligently like the human mind. AI is accomplished by studying the patterns of the human brain and by analyzing the cognitive process. The outcome of these studies develops intelligent software and systems.



Weak AI vs. Strong AI

When discussing artificial intelligence (AI), it is common to distinguish between two broad categories: weak AI and strong AI.

Weak AI (Narrow AI)

Weak AI refers to AI systems that are designed to perform specific tasks and are limited to those tasks only. These AI systems excel at their designated functions

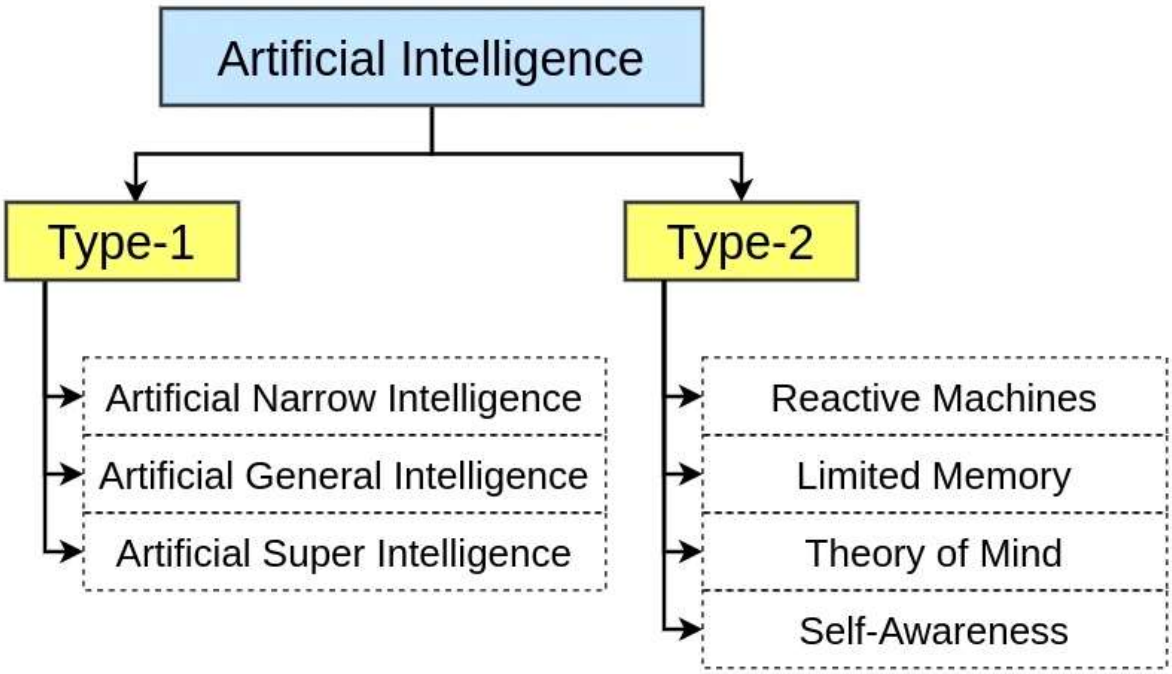
but lack general intelligence. Examples of weak AI include voice assistants like Siri or Alexa, recommendation algorithms, and image recognition systems. Weak AI operates within predefined boundaries and cannot generalize beyond their specialized domain.

Strong AI (General AI)

Strong AI, also known as general AI, refers to AI systems that possess human-level intelligence or even surpass human intelligence across a wide range of tasks. Strong AI would be capable of understanding, reasoning, learning, and applying knowledge to solve complex problems in a manner similar to human cognition. However, the development of strong AI is still largely theoretical and has not been achieved to date.

Strong vs Weak AI Comparison Chart	
Weak AI	**Strong AI**
Weak AI is simply the view that intelligent behavior can be modeled and used by computers to solve complex problems.	Strong AI refers to a hypothetical machine that exhibits human cognitive abilities.
Weak AI refers to systems that are programmed to accomplish a wide range of problems but operate within a predefined range of functions.	Strong AI refers to machines with the mind of their own and which can think and accomplish complex tasks on their own.
Weak AI-powered machines do not have minds of their own.	Strong AI-powered machines can exhibit strong human cognitive abilities.
Alexa and Siri are the best examples of weak AI programs.	Strong AI is a hypothetical concept which does not exist yet in its true form.

Types of Artificial Intelligence



1. Narrow AI (or Weak AI): Specialized AI designed for specific tasks.

2. General AI (or Strong AI): Human-level intelligence, still theoretical.

3. Artificial Superintelligence (ASI): Hypothetical AI surpassing human intellect.

1. Purely Reactive
These machines do not have any memory or data to work with, specializing in just one field of work. For example, in a chess game, the machine observes the moves and makes the best possible decision to win.

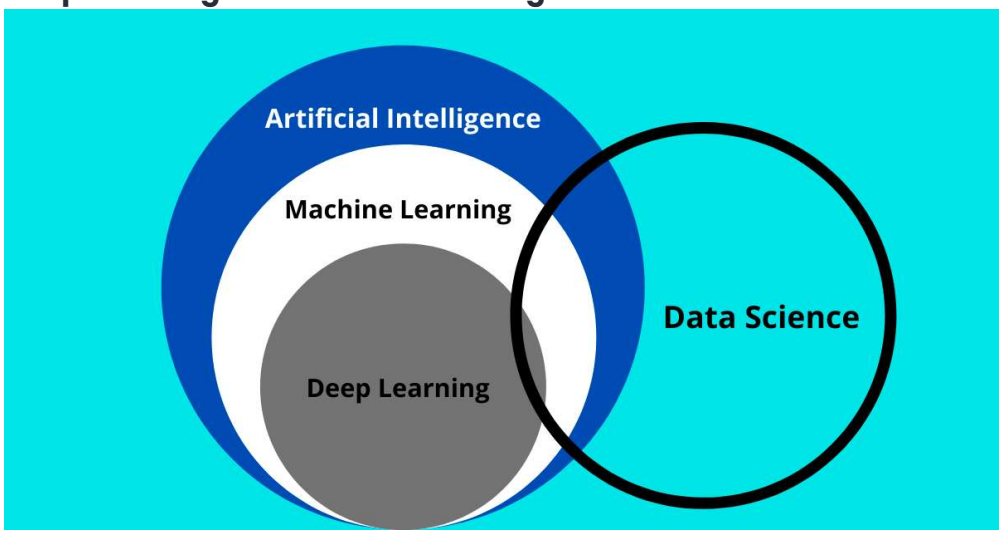
2. Limited Memory
These machines collect previous data and continue adding it to their memory. They have enough memory or experience to make proper decisions, but memory is minimal. For example, this machine can suggest a restaurant based on the location data that has been gathered.

3. Theory of Mind
This kind of AI can understand thoughts and emotions, as well as interact socially. However, a machine based on this type is yet to be built.

4. Self-Aware

Self-aware machines are the future generation of these new technologies . They will be intelligent, sentient, and conscious.

Deep Learning vs. Machine Learning



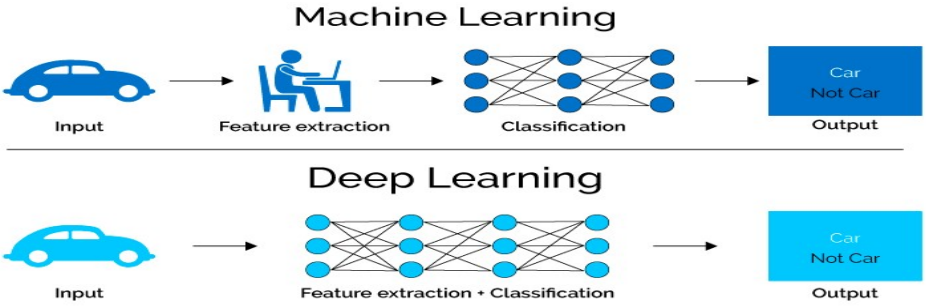
Machine Learning:
Machine Learning focuses on the development of algorithms and models that enable computers to learn from data and make predictions or decisions without explicit programming. Here are key characteristics of machine learning:

1. Feature Engineering: In machine learning, experts manually engineer or select relevant features from the input data to aid the algorithm in making accurate predictions.
2. Supervised and Unsupervised Learning: Machine learning algorithms can be categorized into supervised learning, where models learn from labeled data with known outcomes, and unsupervised learning, where algorithms discover patterns and structures in unlabeled data.
3. Broad Applicability: Machine learning techniques find application across various domains, including image and speech recognition, natural language processing, and recommendation systems.

Deep Learning:
Deep Learning is a subset of machine learning that focuses on training artificial neural networks inspired by the human brain's structure and functioning. Here are key characteristics of deep learning:

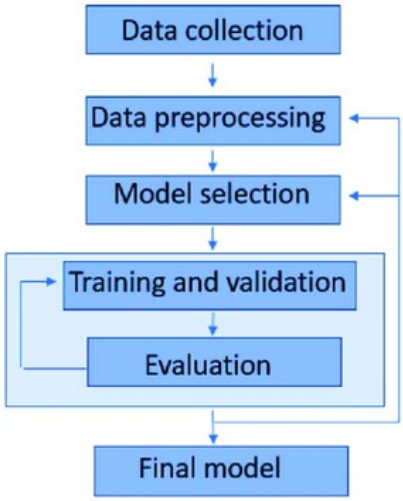
1. Automatic Feature Extraction: Deep learning algorithms have the ability to automatically extract relevant features from raw data, eliminating the need for explicit feature engineering.

2. Deep Neural Networks: Deep learning employs neural networks with multiple layers of interconnected nodes (neurons), enabling the learning of complex hierarchical representations of data.
3. High Performance: Deep learning has demonstrated exceptional performance in domains such as computer vision, natural language processing, and speech recognition, often surpassing traditional machine learning approaches.



How Does Artificial Intelligence Work?

Put simply, AI systems work by merging large with intelligent, iterative processing algorithms. This combination allows AI to learn from patterns and features in the analyzed data. Each time an Artificial Intelligence system performs a round of data processing, it tests and measures its performance and uses the results to develop additional expertise.



Ways of Implementing AI

Machine Learning

It is machine learning that gives AI the ability to learn. This is done by using algorithms to discover patterns and generate insights from the data they are exposed to.

Deep Learning

Deep learning, which is a subcategory of machine learning, provides AI with the ability to mimic a human brain's neural network. It can make sense of patterns, noise, and sources of confusion in the data.

AI Programming Cognitive Skills: Learning, Reasoning and Self-Correction

Artificial Intelligence emphasizes three cognitive skills of learning, reasoning, and self-correction, skills that the human brain possess to one degree or another. We define these in the context of AI as:

- Learning: The acquisition of information and the rules needed to use that information.
- Reasoning: Using the information rules to reach definite or approximate conclusions.
- Self-Correction: The process of continually fine-tuning AI algorithms and ensuring that they offer the most accurate results they can.

However, researchers and programmers have extended and elaborated the goals of AI to the following:

1. Logical Reasoning

   AI programs enable computers to perform sophisticated tasks. On February 10, 1996, IBM's Deep Blue computer won a game of chess against a former world champion, Garry Kasparov.

2. Knowledge Representation

Smalltalk is an object-oriented, dynamically typed, reflective programming language that was created to underpin the "new world" of computing exemplified by "human-computer symbiosis."

3. Planning and Navigation

The process of enabling a computer to get from point A to point B. A prime example of this is Google's self-driving Toyota Prius.

4. Natural Language Processing

Set up computers that can understand and process language.

5. Perception

Use computers to interact with the world through sight, hearing, touch, and smell.

6. Emergent Intelligence

Intelligence that is not explicitly programmed, but emerges from the rest of the specific AI features. The vision for this goal is to have machines exhibit emotional intelligence and moral reasoning.

Some of the tasks performed by AI-enabled devices include:

- Speech recognition
- Object detection
- Solve problems and learn from the given data
- Plan an approach for future tests to be done

Advantages and Disadvantages of AI

Artificial intelligence has its pluses and minuses, much like any other concept or innovation. Here's a quick rundown of some pros and cons.

Pros

- It reduces human error
- It never sleeps, so it's available 24x7
- It never gets bored, so it easily handles repetitive tasks
- It's fast

Cons

- It's costly to implement
- It can't duplicate human creativity
- It will definitely replace some jobs, leading to unemployment
- People can become overly reliant on it

Applications of Artificial Intelligence

Artificial intelligence (AI) has a wide range of applications across various industries and domains. Here are some notable applications of AI:

- Natural Language Processing (NLP)

AI is used in NLP to analyze and understand human language. It powers applications such as speech recognition, machine translation, sentiment analysis, and virtual assistants like Siri and Alexa.

- Image and Video Analysis

AI techniques, including computer vision, enable the analysis and interpretation of images and videos. This finds application in facial recognition, object detection and tracking, content moderation, medical imaging, and autonomous vehicles.

- Robotics and Automation

AI plays a crucial role in robotics and automation systems. Robots equipped with AI algorithms can perform complex tasks in manufacturing, healthcare, logistics, and exploration. They can adapt to changing environments, learn from experience, and collaborate with humans.

- Recommendation Systems

AI-powered recommendation systems are used in e-commerce, streaming platforms, and social media to personalize user experiences. They analyze user preferences, behavior, and historical data to suggest relevant products, movies, music, or content.

- Financial Services

AI is extensively used in the finance industry for fraud detection, algorithmic trading, credit scoring, and risk assessment. Machine learning models can analyze vast amounts of financial data to identify patterns and make predictions.

- Healthcare

AI applications in healthcare include disease diagnosis, medical imaging analysis, drug discovery, personalized medicine, and patient monitoring. AI can assist in identifying patterns in medical data and provide insights for better diagnosis and treatment.

- Virtual Assistants and Chatbots

AI-powered virtual assistants and chatbots interact with users, understand their queries, and provide relevant information or perform tasks. They are used in customer support, information retrieval, and personalized assistance.

- Gaming

AI algorithms are employed in gaming for creating realistic virtual characters, opponent behavior, and intelligent decision-making. AI is also used to optimize game graphics, physics simulations, and game testing.

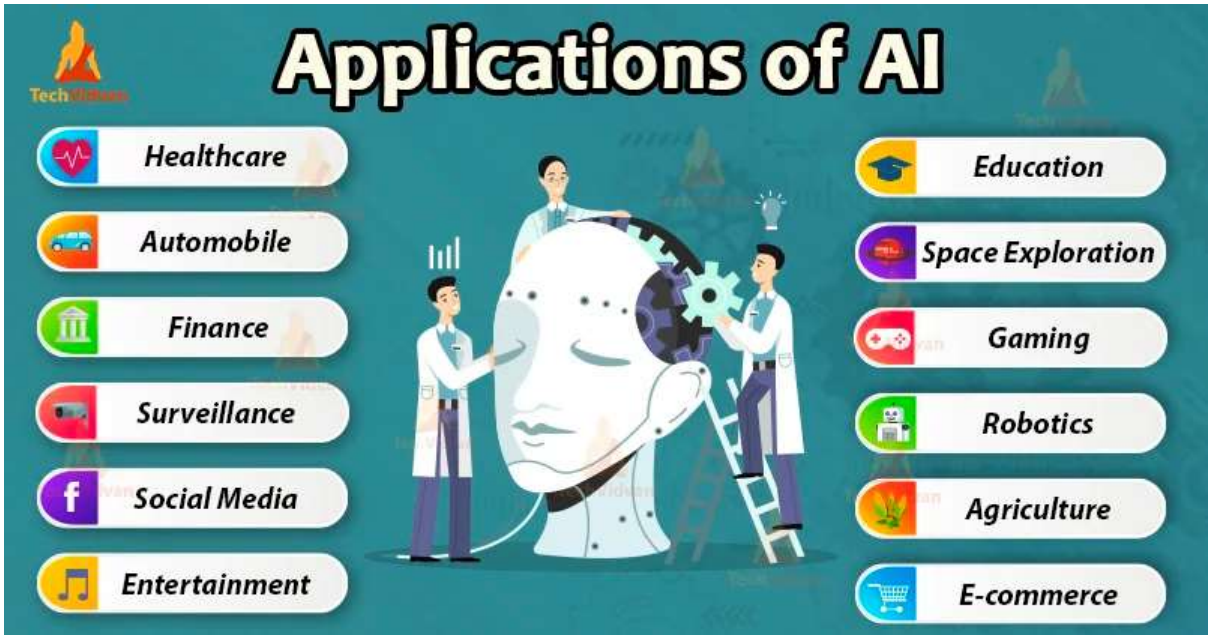
- Smart Homes and IoT

AI enables the development of smart home systems that can automate tasks, control devices, and learn from user preferences. AI can enhance the functionality and efficiency of Internet of Things (IoT) devices and networks.

- Cybersecurity

AI helps in detecting and preventing cyber threats by analyzing network traffic, identifying anomalies, and predicting potential attacks. It can enhance the security of systems and data through advanced threat detection and response mechanisms.

These are just a few examples of how AI is applied in various fields. The potential of AI is vast, and its applications continue to expand as technology advances.



Artificial Intelligence Examples

Artificial Intelligence (AI) has become an integral part of our daily lives, revolutionizing various industries and enhancing user experiences. Here are some notable examples of AI applications:

ChatGPT

ChatGPT is an advanced language model developed by OpenAI, capable of generating human-like responses and engaging in natural language conversations. It uses deep learning techniques to understand and generate coherent text, making it useful for customer support, chatbots, and virtual assistants.

Google Maps

Google Maps utilizes AI algorithms to provide real-time navigation, traffic updates, and personalized recommendations. It analyzes vast amounts of data, including historical traffic patterns and user input, to suggest the fastest routes, estimate arrival times, and even predict traffic congestion.

Smart Assistants

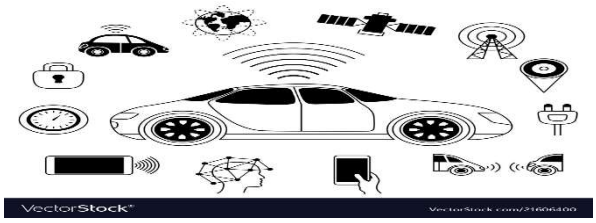
Smart assistants like Amazon's Alexa, Apple's Siri, and Google Assistant employ AI technologies to interpret voice commands, answer questions, and perform tasks. These assistants use natural language processing and machine learning algorithms to understand user intent, retrieve relevant information, and carry out requested actions.

Snapchat Filters

Snapchat

Snapchat's augmented reality filters, or "Lenses," incorporate AI to recognize facial features, track movements, and overlay interactive effects on users' faces in real-time. AI algorithms enable Snapchat to apply various filters, masks, and animations that align with the user's facial expressions and movements.

Self-Driving Cars



Self-driving cars rely heavily on AI for perception, decision-making, and control. Using a combination of sensors, cameras, and machine learning algorithms, these vehicles can detect objects, interpret traffic signs, and navigate complex road conditions autonomously, enhancing safety and efficiency on the roads.

Wearables

Wearable devices, such as fitness trackers and smartwatches, utilize AI to monitor and analyze users' health data. They track activities, heart rate, sleep patterns,

and more, providing personalized insights and recommendations to improve overall well-being.

MuZero
MuZero is an AI algorithm developed by DeepMind that combines reinforcement learning and deep neural networks. It has achieved remarkable success in playing complex board games like chess, Go, and shogi at a superhuman level. MuZero learns and improves its strategies through self-play and planning.
These examples demonstrate the wide-ranging applications of AI, showcasing its potential to enhance our lives, improve efficiency, and drive innovation across various industries.

FAQs

1. Where is AI used?
Artificial intelligence is frequently utilized to present individuals with personalized suggestions based on their prior searches and purchases and other online behavior. AI is extremely crucial in commerce, such as product optimization, inventory planning, and logistics. Machine learning, cybersecurity, customer relationship management, internet searches, and personal assistants are some of the most common applications of AI. Voice assistants, picture recognition for face unlocking in cell phones, and ML-based financial fraud detection are all examples of AI software that is now in use.

2. What is artificial intelligence in simple words?
Artificial Intelligence (AI) in simple words refers to the ability of machines or computer systems to perform tasks that typically require human intelligence. It is a field of study and technology that aims to create machines that can learn from experience, adapt to new information, and carry out tasks without explicit programming. Artificial Intelligence (AI) refers to the simulation of human intelligence in machines that are programmed to think like humans and mimic their actions.

3. What Are the 4 Types of AI?
The current categorization system categorizes AI into four basic categories: reactive, theory of mind, limited memory, and self-aware.

4. How Is AI Used Today?
Machines today can learn from experience, adapt to new inputs, and even perform human-like tasks with help from artificial intelligence (AI). Artificial

intelligence examples today, from chess-playing computers to self-driving cars, are heavily based on deep learning and natural language processing. There are several examples of AI software in use in daily life, including voice assistants, face recognition for unlocking mobile phones and machine learning-based financial fraud detection. AI software is typically obtained by downloading AI-capable software from an internet marketplace, with no additional hardware required.

5. What are some examples of AI in everyday life?

Examples of AI in everyday life include voice assistants like Siri or Alexa, recommendation engines like Netflix's movie recommendations, and autonomous vehicles.

6. How is AI helping in our life?

AI and ML-powered software and gadgets mimic human brain processes to assist society in advancing with the digital revolution. AI systems perceive their environment, deal with what they observe, resolve difficulties, and take action to help with duties to make daily living easier. People check their social media accounts on a frequent basis, including Facebook, Twitter, Instagram, and other sites. AI is not only customizing your feeds behind the scenes, but it is also recognizing and deleting bogus news. So, AI is assisting you in your daily life.

7. What are the three types of AI?

The three types of AI are:

1. Artificial Narrow Intelligence (ANI): Also known as Weak AI, it specializes in performing specific tasks and lacks general cognitive abilities.
2. Artificial General Intelligence (AGI): Refers to Strong AI capable of understanding, learning, and applying knowledge across various domains, similar to human intelligence.
3. Artificial Superintelligence (ASI): Hypothetical AI surpassing human intelligence in all aspects, potentially capable of solving complex problems and making advancements beyond human comprehension.

8. Is AI dangerous?

Aside from planning for a future with super-intelligent computers, artificial intelligence in its current state might already offer problems.

9. What are the advantages of AI?

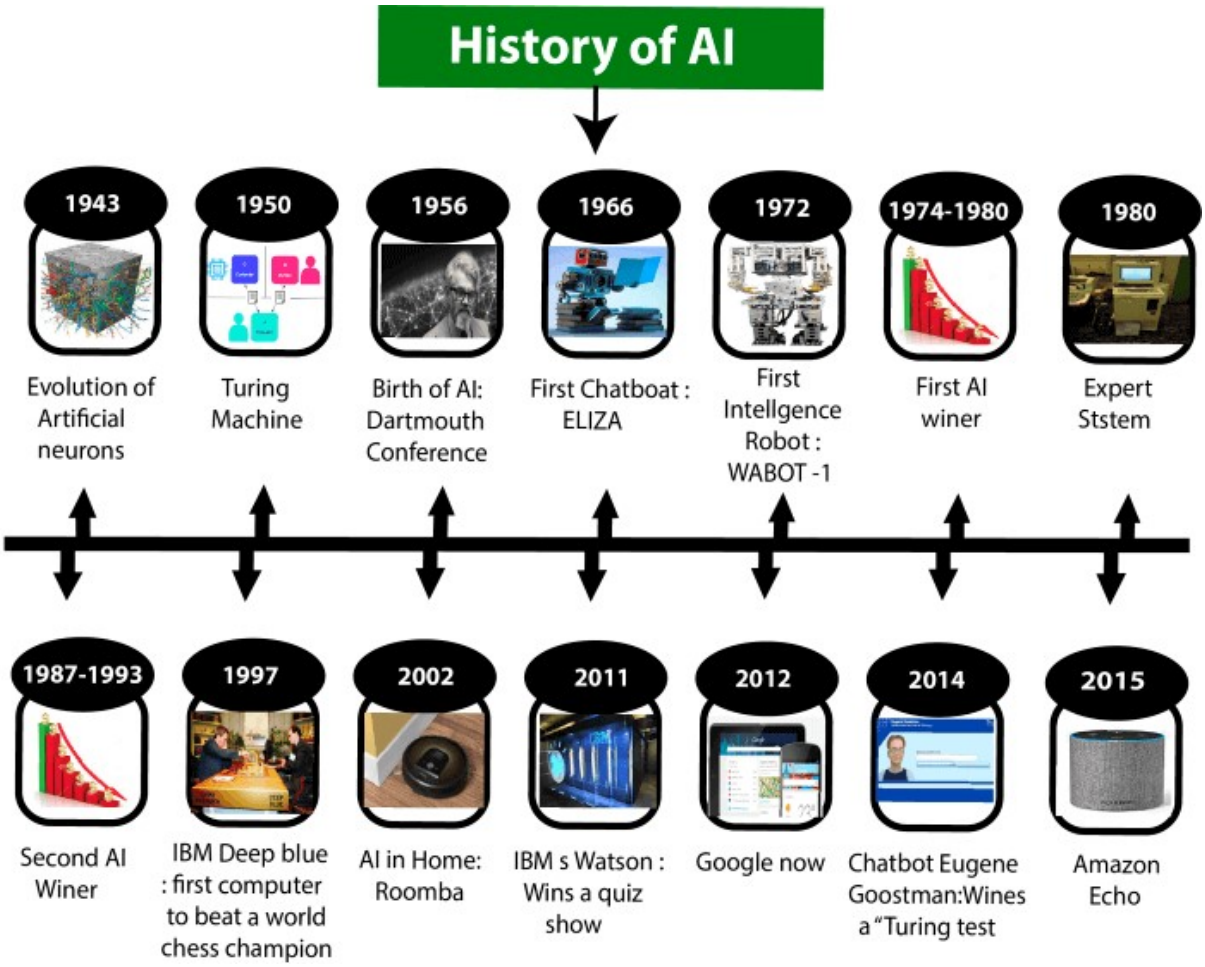
The advantages of AI include reducing the time it takes to complete a task, reducing the cost of previously done activities, continuously and without interruption, with no downtime, and improving the capacities of people with disabilities.

10. What are the 7 main areas of AI?

The main seven areas of AI are:

1. Machine Learning: Involves algorithms that enable machines to learn from data and improve their performance without explicit programming.
2. Natural Language Processing (NLP): Focuses on enabling computers to understand, interpret, and generate human language.
3. Computer Vision: Deals with giving machines the ability to interpret and understand visual information from images or videos.
4. Robotics: Combines AI and mechanical engineering to create intelligent machines capable of performing tasks autonomously.
5. Expert Systems: Utilizes knowledge and reasoning to solve complex problems in specific domains, mimicking human expertise.
6. Speech Recognition: Involves converting spoken language into text or commands, enabling machines to interact with users through speech.
7. Planning and Decision Making: Focuses on algorithms that allow AI systems to make choices and optimize actions to achieve specific goals.

History of AI



Artificial intelligence, or at least the modern concept of it, has been with us for several decades, but only in the recent past has AI captured the collective psyche of everyday business and society.

The introduction of AI in the 1950s very much paralleled the beginnings of the Atomic Age. Though their evolutionary paths have differed, both technologies are viewed as posing an existential threat to humanity.

Perceptions about the darker side of AI aside, artificial intelligence tools and technologies, since the advent of the Turing test in 1950 have made incredible strides -- despite the intermittent roller-coaster rides mainly due to funding fits and starts for AI research. Many of these breakthrough advancements have flown under the radar, visible mostly to academic, government and scientific research circles until the past decade or so, when AI was practically applied to the wants and needs of the masses. AI products such as Apple's Siri and Amazon's Alexa,

online shopping, social media feeds and self-driving cars have forever altered the lifestyles of consumers and operations of businesses.

Through the decades, some of the more notable developments include the following:

- Neural networks and the coining of the terms *artificial intelligence* and *machine learning* in the 1950s.
- Eliza, the chatbot with cognitive capabilities, and Shakey, the first mobile intelligent robot, in the 1960s.
- AI winter followed by AI renaissance in the 1970s and 1980s.
- Speech and video processing in the 1990s.
- IBM Watson, personal assistants, facial recognition, deepfakes, autonomous vehicles, and content and image creation in the 2000s.

1950

Alan Turing published "Computing Machinery and Intelligence," introducing the Turing test and opening the doors to what would be known as AI.

1951

Marvin Minsky and Dean Edmonds developed the first artificial neural network (ANN) called SNARC using 3,000 vacuum tubes to simulate a network of 40 neurons.

1952

Arthur Samuel developed Samuel Checkers-Playing Program, the world's first program to play games that was self-learning.

1956

John McCarthy, Marvin Minsky, Nathaniel Rochester and Claude Shannon coined the term *artificial intelligence* in a proposal for a workshop widely recognized as a founding event in the AI field.

1958

Frank Rosenblatt developed the perceptron, an early ANN that could learn from data and became the foundation for modern neural networks.

John McCarthy developed the programming language Lisp, which was quickly adopted by the AI industry and gained enormous popularity among developers.

1959

Arthur Samuel coined the term *machine learning* in a seminal paper explaining that the computer could be programmed to outplay its programmer.

Oliver Selfridge published "Pandemonium: A Paradigm for Learning," a landmark contribution to machine learning that described a model that could adaptively improve itself to find patterns in events.

1964

Daniel Bobrow developed STUDENT, an early natural language processing (NLP) program designed to solve algebra word problems, while he was a doctoral candidate at MIT.

1965

Edward Feigenbaum, Bruce G. Buchanan, Joshua Lederberg and Carl Djerassi developed the first expert system, Dendral, which assisted organic chemists in identifying unknown organic molecules.

1966

Joseph Weizenbaum created Eliza, one of the more celebrated computer programs of all time, capable of engaging in conversations with humans and making them believe the software had humanlike emotions.

Stanford Research Institute developed Shakey, the world's first mobile intelligent robot that combined AI, computer vision, navigation and NLP. It's the grandfather of self-driving cars and drones.

1968

Terry Winograd created SHRDLU, the first multimodal AI that could manipulate and reason out a world of blocks according to instructions from a user.

1969

Arthur Bryson and Yu-Chi Ho described a backpropagation learning algorithm to enable multilayer ANNs, an advancement over the perceptron and a foundation for deep learning.

Marvin Minsky and Seymour Papert published the book *Perceptrons*, which described the limitations of simple neural networks and caused neural network research to decline and symbolic AI research to thrive.

1973

James Lighthill released the report "Artificial Intelligence: A General Survey," which caused the British government to significantly reduce support for AI research.

1980

Symbolics Lisp machines were commercialized, signaling an AI renaissance. Years later, the Lisp machine market collapsed.

1981

Danny Hillis designed parallel computers for AI and other computational tasks, an architecture similar to modern GPUs.

1984

Marvin Minsky and Roger Schank coined the term *AI winter* at a meeting of the Association for the Advancement of Artificial Intelligence, warning the business community that AI hype would lead to disappointment and the collapse of the industry, which happened three years later.

1985

Judea Pearl introduced Bayesian networks causal analysis, which provides statistical techniques for representing uncertainty in computers.

1988

Peter Brown et al. published "A Statistical Approach to Language Translation," paving the way for one of the more widely studied machine translation methods.

1989

Yann LeCun, Yoshua Bengio and Patrick Haffner demonstrated how convolutional neural networks (CNNs) can be used to recognize handwritten characters, showing that neural networks could be applied to real-world problems.

Neural networks have differing characteristics.

1997

Sepp Hochreiter and Jürgen Schmidhuber proposed the Long Short-Term Memory recurrent neural network, which could process entire sequences of data such as speech or video.

IBM's Deep Blue defeated Garry Kasparov in a historic chess rematch, the first defeat of a reigning world chess champion by a computer under tournament conditions.

2000

University of Montreal researchers published "A Neural Probabilistic Language Model," which suggested a method to model language using feedforward neural networks.

2006

Fei-Fei Li started working on the ImageNet visual database, introduced in 2009, which became a catalyst for the AI boom and the basis of an annual competition for image recognition algorithms.

IBM Watson originated with the initial goal of beating a human on the iconic quiz show *Jeopardy!* In 2011, the question-answering computer system defeated the show's all-time (human) champion, Ken Jennings.

2009

Rajat Raina, Anand Madhavan and Andrew Ng published "Large-Scale Deep Unsupervised Learning Using Graphics Processors," presenting the idea of using GPUs to train large neural networks.

2011

Jürgen Schmidhuber, Dan Claudiu Cireșan, Ueli Meier and Jonathan Masci developed the first CNN to achieve "superhuman" performance by winning the German Traffic Sign Recognition competition.

Apple released Siri, a voice-powered personal assistant that can generate responses and take actions in response to voice requests.

2012

Geoffrey Hinton, Ilya Sutskever and Alex Krizhevsky introduced a deep CNN architecture that won the ImageNet challenge and triggered the explosion of deep learning process and implementation.

2013

China's Tianhe-2 doubled the world's top supercomputing speed at 33.86 petaflops, retaining the title of the world's fastest system for the third consecutive time.

DeepMind introduced deep reinforcement learning, a CNN that learned based on rewards and learned to play games through repetition, surpassing human expert levels.

Google researcher Tomas Mikolov and colleagues introduced Word2vec to automatically identify semantic relationships between words.

2014

Ian Goodfellow and colleagues invented generative adversarial networks, a class of machine learning frameworks used to generate photos , transform images and create deepfakes.

Diederik Kingma and Max Welling introduced variational autoencoders to generate images, videos and text.

Facebook developed the deep learning facial recognition system DeepFace, which identifies human faces in digital images with near-human accuracy.

2016

DeepMind's AlphaGo defeated top Go player Lee Sedol in Seoul, South Korea, drawing comparisons to the Kasparov chess match with Deep Blue nearly 20 years earlier.

Uber started a self-driving car pilot program in Pittsburgh for a select group of users.

Five pillars of AI technology changing the way businesses operate.

2017

Stanford researchers published work on diffusion models in the paper "Deep Unsupervised Learning Using Nonequilibrium Thermodynamics." The technique provides a way to reverse-engineer the process of adding noise to a final image.

Google researchers developed the concept of transformers in the seminal paper "Attention Is All You Need," inspiring subsequent research into tools that could automatically parse unlabeled text into large language models (LLMs).

British physicist Stephen Hawking warned, "Unless we learn how to prepare for, and avoid, the potential risks, AI could be the worst event in the history of our civilization."

2018

Developed by IBM, Airbus and the German Aerospace Center DLR, Cimon was the first robot sent into space to assist astronauts.

OpenAI released GPT (Generative Pre-trained Transformer), paving the way for subsequent LLMs.

Groove X unveiled a home mini-robot called Lovot that could sense and affect mood changes in humans.

2019

Microsoft launched the Turing Natural Language Generation generative language model with 17 billion parameters.

Google AI and Langone Medical Center's deep learning algorithm outperformed radiologists in detecting potential lung cancers.

2020

The University of Oxford developed an AI test called Curial to rapidly identify COVID-19 in emergency room patients.

Nvidia announced the beta version of its Omniverse platform to create 3D models in the physical world.

DeepMind's AlphaFold system won the Critical Assessment of Protein Structure Prediction protein-folding contest.

2021

OpenAI introduced the Dall-E multimodal AI system that can generate images from text prompts.

The University of California, San Diego, created a four-legged soft robot that functioned on pressurized air instead of electronics.

2022

Google software engineer Blake Lemoine was fired for revealing secrets of Lamda and claiming it was sentient.

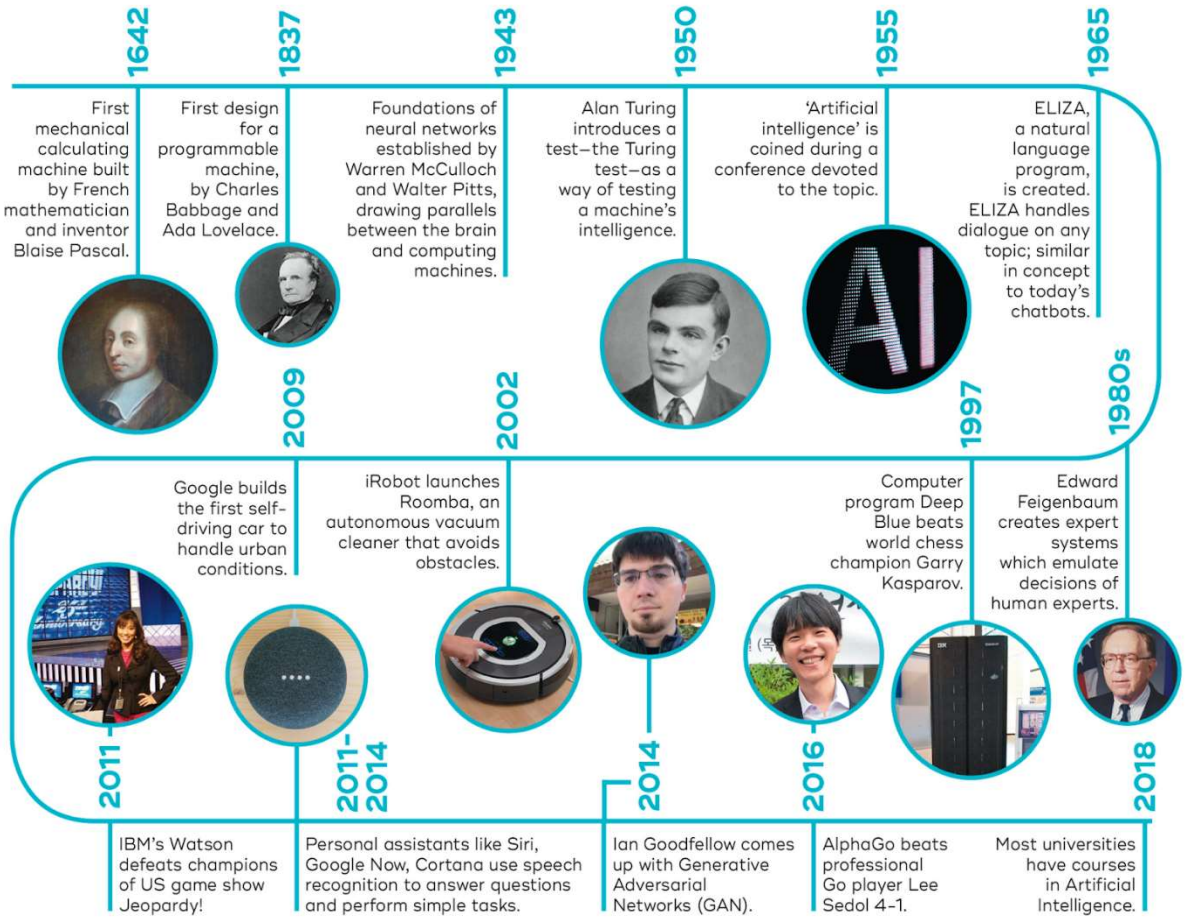
DeepMind unveiled AlphaTensor "for discovering novel, efficient and provably correct algorithms".

Intel claimed its FakeCatcher real-time deepfake detector was 96% accurate.

OpenAI released ChatGPT in November to provide a chat-based interface to its GPT-3.5 LLM.

2023

OpenAI announced the GPT-4 multimodal LLM that receives both text and image prompts.



Beyond 2023

We can only begin to envision AI's continuing technological advancements and influences in business processes, manufacturing, healthcare, financial services, marketing, customer experience, workforce environments, education, agriculture, law, IT systems and management, cybersecurity, and ground, air and space transportation.

In business, 55% of organizations that have deployed AI always consider AI for every new use case they're evaluating, according to a 2023 Gartner survey. By 2026, Gartner reported, organizations that "operationalize AI transparency, trust and security will see their AI models achieve a 50% improvement in terms of adoption, business goals and user acceptance."

Today's tangible developments -- some incremental, some disruptive -- are advancing AI's ultimate goal of achieving artificial general intelligence. Along these lines, neuromorphic processing shows promise in mimicking human brain cells, enabling computer programs to work simultaneously instead of sequentially. Amid these and other mind-boggling advancements, issues of trust, privacy, transparency, accountability, ethics and humanity have emerged and

will continue to clash and seek levels of acceptability among business and society.

Turing Test in Artificial Intelligence

The Turing test was developed by Alan Turing(A computer scientist) in 1950. He proposed that the "Turing test is used to determine whether or not a computer(machine) can think intelligently like humans"?

The Turing Test is a widely used measure of a machine's ability to demonstrate human-like intelligence. It was first proposed by British mathematician and computer scientist Alan Turing in 1950.

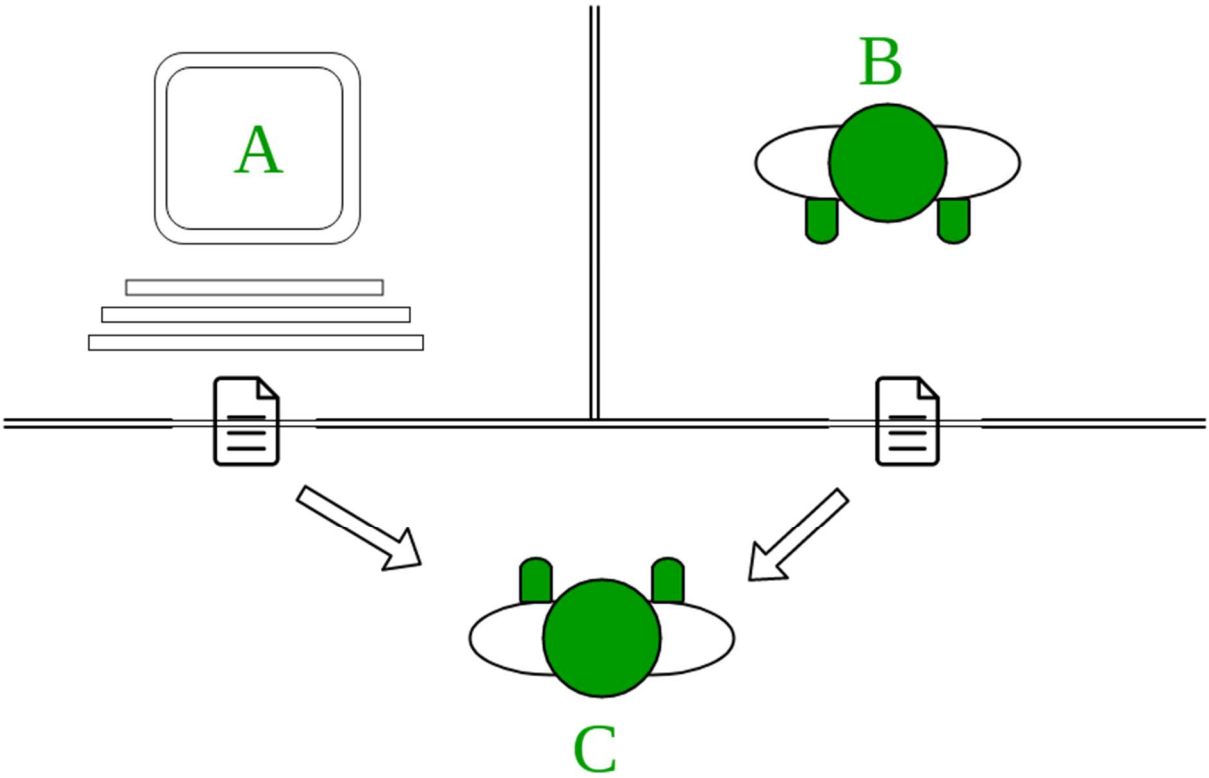
The basic idea of the Turing Test is simple: a human judge engages in a text-based conversation with both a human and a machine, and then decides which of the two they believe to be a human. If the judge is unable to distinguish between the human and the machine based on the conversation, then the machine is said to have passed the Turing Test.

The Turing Test is widely used as a benchmark for evaluating the progress of artificial intelligence research, and has inspired numerous studies and experiments aimed at developing machines that can pass the test.

While the Turing Test has been used as a measure of machine intelligence for over six decades, it is not without its critics. Some argue that the test is too focused on language and does not take into account other important aspects of intelligence, such as perception, problem-solving, and decision-making.

Despite its limitations, the Turing Test remains an important reference point in the field of artificial intelligence and continues to inspire new research and development in this area.

Imagine a game of three players having two humans and one computer, an interrogator(as a human) is isolated from the other two players. The interrogator's job is to try and figure out which one is human and which one is a computer by asking questions from both of them. To make things harder computer is trying to make the interrogator guess wrongly. In other words, computers would try to be indistinguishable from humans as much as possible.



The “standard interpretation” of the Turing Test, in which player C, the interrogator, is given the task of trying to determine which player – A or B – is a computer and which is a human. The interrogator is limited to using the responses to written questions to make the determination

The conversation between interrogator and computer would be like this:

C(Interrogator): Are you a computer?

A(Computer): No

C: Multiply one large number to another, 158745887 * 56755647

A: After a long pause, an incorrect answer!

C: Add 5478012, 4563145

A: (Pause about 20 seconds and then give an answer)10041157

If the interrogator wouldn’t able to distinguish the answers provided by both humans and computers then the computer passes the test and the machine(computer) is considered as intelligent as a human. In other words, a computer would be considered intelligent if its conversation couldn’t be easily distinguished from a human’s. The whole conversation would be limited to a text-only channel such as a computer keyboard and screen.

He also proposed that by the year 2000 a computer “would be able to play the imitation game so well that an average interrogator will not have more than a 70-percent chance of making the right identification (machine or human) after five minutes of questioning.” No computer has come close to this standard.

But in the year 1980, Mr. John Searle proposed the “Chinese room argument“. He argued that the Turing test could not be used to determine “whether or not a machine is considered as intelligent like humans”. He argued that any machine

like ELIZA and PARRY could easily pass the Turing Test simply by manipulating symbols of which they had no understanding. Without understanding, they could not be described as “thinking” in the same sense people do.

Advantages of the Turing Test in Artificial Intelligence:

1. Evaluating machine intelligence: The Turing Test provides a simple and well-known method for evaluating the intelligence of a machine.
2. Setting a benchmark: The Turing Test sets a benchmark for artificial intelligence research and provides a goal for researchers to strive towards.
3. Inspiring research: The Turing Test has inspired numerous studies and experiments aimed at developing machines that can pass the test, which has driven progress in the field of artificial intelligence.
4. Simple to administer: The Turing Test is relatively simple to administer and can be carried out with just a computer and a human judge.

Disadvantages of the Turing Test in Artificial Intelligence:

1. Limited scope: The Turing Test is limited in scope, focusing primarily on language-based conversations and not taking into account other important aspects of intelligence, such as perception, problem-solving, and decision-making.
2. Human bias: The results of the Turing Test can be influenced by the biases and preferences of the human judge, making it difficult to obtain objective and reliable results.
3. Not representative of real-world AI: The Turing Test may not be representative of the kind of intelligence that machines need to demonstrate in real-world applications.

What is Data Science?

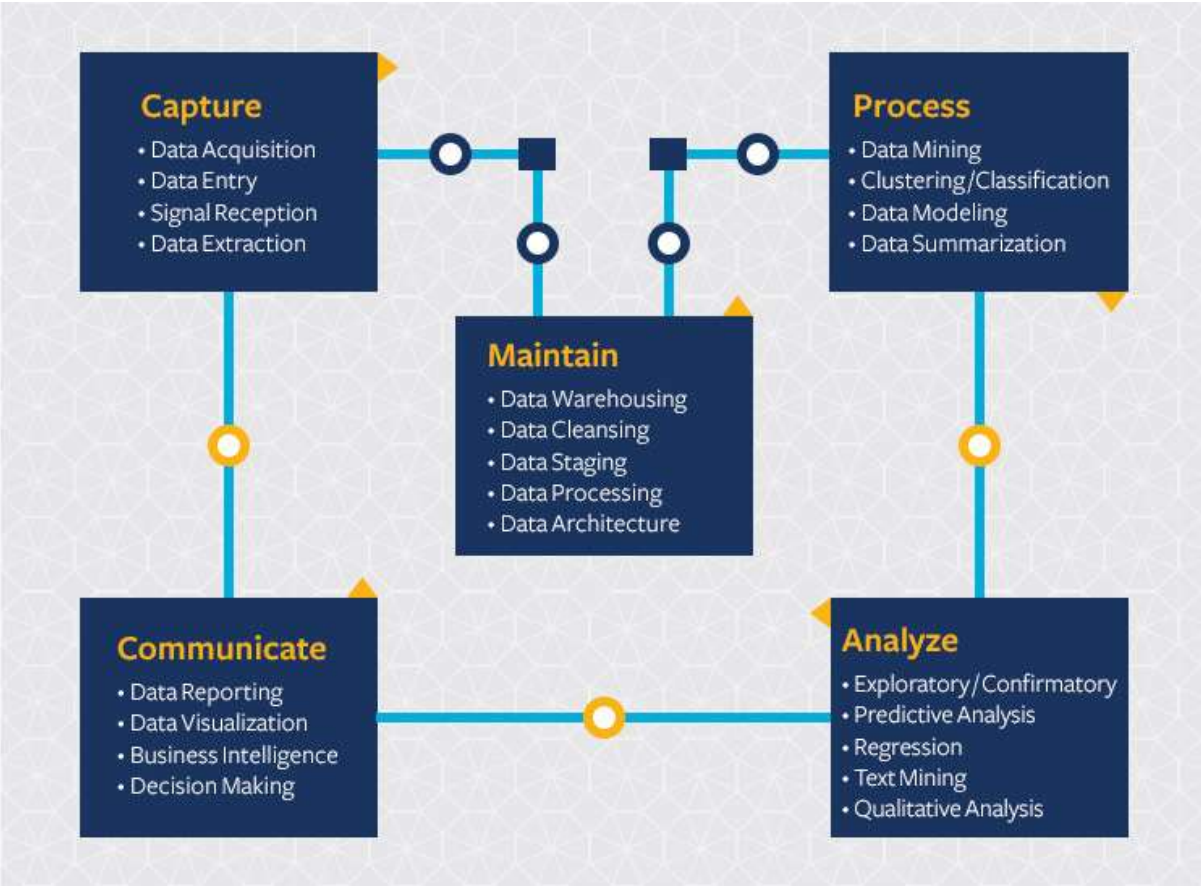
Data science combines math and statistics, specialized programming, advanced analytics, artificial intelligence (AI), and machine learning with specific subject matter expertise to uncover actionable insights hidden in an organization's data. These insights can be used to guide decision making and strategic planning.

The accelerating volume of data sources, and subsequently data, has made data science is one of the fastest growing field across every industry. Organizations are increasingly reliant on them to interpret data and provide actionable recommendations to improve business outcomes.

The data science lifecycle involves various roles, tools, and processes, which enables analysts to glean actionable insights. Typically, a data science project undergoes the following stages:

- Data ingestion: The lifecycle begins with the data collection--both raw structured and unstructured data from all relevant sources using a variety of methods. These methods can include manual entry, web scraping, and real-time streaming data from systems and devices. Data sources can include structured data, such as customer data, along with unstructured data like log files, video, audio, pictures, the Internet of Things (IoT), social media, and more.
- Data storage and data processing: Since data can have different formats and structures, companies need to consider different storage systems based on the type of data that needs to be captured. Data management teams help to set standards around data storage and structure, which facilitate workflows around analytics, machine learning and deep learning models. This stage includes cleaning data, deduplicating, transforming and combining the data using ETL (extract, transform, load) jobs or other data integration technologies. This data preparation is essential for promoting data quality before loading into a data warehouse, data lake, or other repository.
- Data analysis: Here, data scientists conduct an exploratory data analysis to examine biases, patterns, ranges, and distributions of values within the data. This data analytics exploration drives hypothesis generation for a/b testing. It also allows analysts to determine the data's relevance for use within modeling efforts for predictive analytics, machine learning, and/or deep learning. Depending on a model's accuracy, organizations can become reliant on these insights for business decision making, allowing them to drive more scalability.

- Communicate: Finally, insights are presented as reports and other data visualizations that make the insights—and their impact on business—easier for business analysts and other decision-makers to understand. A data science programming language such as R or Python includes components for generating visualizations; alternately, data scientists can use dedicated visualization tools.



The image represents the five stages of the data science life cycle: ***Capture****, (data acquisition, data entry, signal reception, data extraction);* ***Maintain*** *(data warehousing, data cleansing, data staging, data processing, data architecture);* ***Process*** *(data mining, clustering/classification, data modeling, data summarization);* ***Analyze*** *(exploratory/confirmatory, predictive analysis, regression, text mining, qualitative analysis);* ***Communicate*** *(data reporting, data visualization, business intelligence, decision making).*

Data science versus data scientist

Data science is considered a discipline, while data scientists are the practitioners within that field. Data scientists are not necessarily directly responsible for all the processes involved in the data science lifecycle. For example, data pipelines are typically handled by data engineers—but the data scientist may make recommendations about what sort of data is useful or required. While data scientists can build machine learning models, scaling these efforts at a larger

level requires more software engineering skills to optimize a program to run more quickly. As a result, it's common for a data scientist to partner with machine learning engineers to scale machine learning models.

Data scientist responsibilities can commonly overlap with a data analyst, particularly with exploratory data analysis and data visualization. However, a data scientist's skillset is typically broader than the average data analyst. Comparatively speaking, data scientist leverage common programming languages, such as R and Python, to conduct more statistical inference and data visualization.

To perform these tasks, data scientists require computer science and pure science skills beyond those of a typical business analyst or data analyst. The data scientist must also understand the specifics of the business, such as automobile manufacturing, eCommerce, or healthcare.

In short, a data scientist must be able to:

- Know enough about the business to ask pertinent questions and identify business pain points.
- Apply statistics and computer science, along with business acumen, to data analysis.
- Use a wide range of tools and techniques for preparing and extracting data—everything from databases and SQL to data mining to data integration methods.
- Extract insights from big data using predictive analytics and artificial intelligence (AI), including machine learning models, natural language processing, and deep learning.
- Write programs that automate data processing and calculations.
- Tell—and illustrate—stories that clearly convey the meaning of results to decision-makers and stakeholders at every level of technical understanding.
- Explain how the results can be used to solve business problems.
- Collaborate with other data science team members, such as data and business analysts, IT architects, data engineers, and application developers.

Data science tools

Data scientists rely on popular programming languages to conduct exploratory data analysis and statistical regression. These open source tools support pre-built statistical modelling, machine learning, and graphics capabilities. These languages include the following :

- R Studio: An open source programming language and environment for developing statistical computing and graphics.
- Python: It is a dynamic and flexible programming language. The Python includes numerous libraries, such as NumPy, Pandas, Matplotlib, for analyzing data quickly.

To facilitate sharing code and other information, data scientists may use GitHub and Jupyter notebooks.

Here are a few representative use cases for data science and artificial intelligence:

- An international bank delivers faster loan services with a mobile app using machine learning-powered credit risk models and a hybrid cloud computing architecture that is both powerful and secure.
- An electronics firm is developing ultra-powerful 3D-printed sensors to guide tomorrow's driverless vehicles. The solution relies on data science and analytics tools to enhance its real-time object detection capabilities.
- A robotic process automation (RPA) solution provider developed a cognitive business process mining solution that reduces incident handling times between 15% and 95% for its client companies. The solution is trained to understand the content and sentiment of customer emails, directing service teams to prioritize those that are most relevant and urgent.
- A digital media technology company created an audience analytics platform that enables its clients to see what's engaging TV audiences as they're offered a growing range of digital channels. The solution employs deep analytics and machine learning to gather real-time insights into viewer behavior.
- An urban police department created statistical incident analysis tools to help officers understand when and where to deploy resources in order to prevent crime. The data-driven solution creates reports and dashboards to augment situational awareness for field officers.
- Shanghai Changjiang Science and Technology Development used IBM® Watson® technology to build an AI-based medical assessment platform that can analyze existing medical records to categorize patients based on their risk of experiencing a stroke and that can predict the success rate of different treatment plans.

Distance function :

A distance function provides distance between the elements of a set.

A metric or distance function is a function d which takes pairs of points or objects to real numbers and satisfies the following rules:

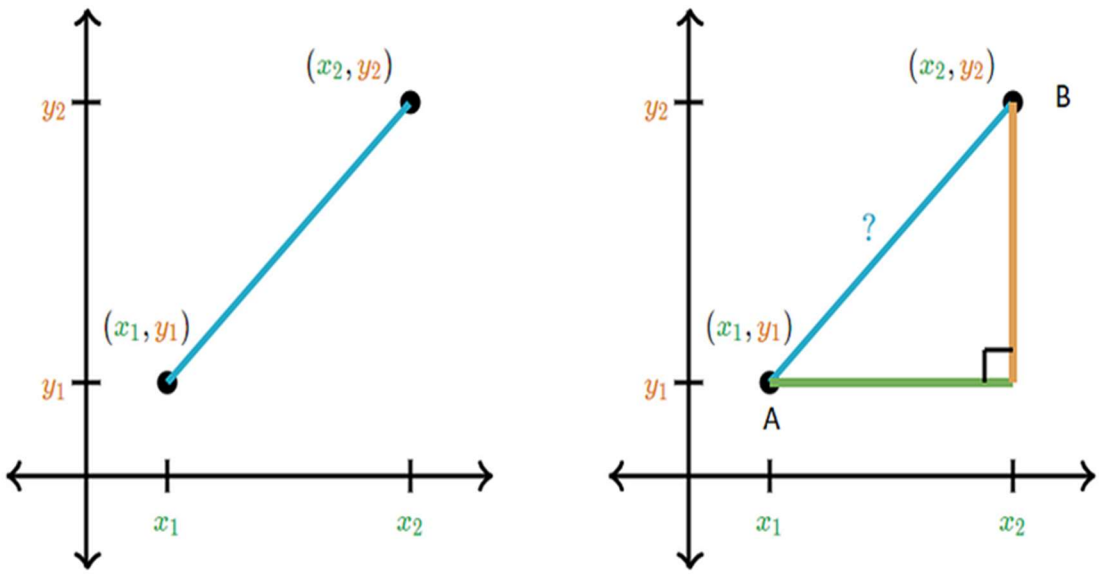
· The distance between an object and itself is always zero.

· The distance between distinct objects is always positive.

· Distance is symmetric: the distance from *x* to *y* is always the same as the distance from *y* to *x*.

d(x,y)=d(y,x) for all x,y

· Distance satisfies the triangle inequality: if *x*, *y*, and *z* are three objects, then d(x,z)≤d(x,y)+d(y,z)



In order to calculate the distance between data points A and B Pythagorean theorem considers the length of x and y axis.

$$? = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

A distance function provides distance between the elements of a set. If the distance is zero then elements are equivalent else they are different from each other.

Distance function(vector norm):
Let's take a n-dimensional vector

$$X=\begin{bmatrix} x_1 \\ x_2 \\ . \\ . \\ x_n \end{bmatrix}$$

A general vector norm |x|, sometimes written as ‖x‖ is a non-negative norm defined as

- $|x|>0$ when $x\neq 0$, and $|x|=0$ iff $x=0$
- $|kx|=|k||x|$ for any scalar k
- $|x+y|\leq|x|+|y|$

The vector norm $|x|_p$ p=1,2,…. Is defined as

$$|x|_p=\left(\sum_i |x_i|^P\right)^{1/P}$$

Some examples of different norms---
X=(1,2,3)

Name	Symbol	value	Approx
L^1-norm	$\|x\|_1$	6	6.000
L^2-norm	$\|x\|_2$	$\sqrt{14}$	3.742
L^3-norm	$\|x\|_3$	$\sqrt[3]{36}$	3.302
L^4-norm	$\|x\|_4$	$\sqrt[4]{98}$	3.147

Euclidean Norm

The most commonly encountered vector norm(often simply called the norm of a vector or sometimes called the magnitude of a vector) is the L2 norm given by

$|x|_2 = |x| = \sqrt{x_1^2 + x_2^2 + \cdots + x_n^2}$

It is commonly known as **Euclidean Norm**. $\mathbb{R}^n$
The n-dimensional Euclidean space ,the intuitive notion of length of the vector $x=(x_1,x_2,\ldots,x_n)$ is

$$\|x\| = \sqrt{x_1^2 + x_2^2 + \cdots + x_n^2}$$

This is the **Euclidean norm**, which gives the ordinary distance from the origin to the point X — a consequence of the Pythagorean theorem.

Distance between two points in 2D

If the points $X=(x1, y1)$ and $Y=(x2, y2)$ are in 2-dimensional space, then the Euclidean distance between them is

$$d(X,Y) = \|X - Y\| = \sqrt{(x2 - x1)^2 + (y2 - y1)^2} \qquad (P=2)$$

Distance between two points in 3D

If the points in 3-dimensional space are

$$X = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$$

$$Y = \begin{bmatrix} Y_1 \\ Y_2 \\ Y_3 \end{bmatrix}$$

then the distance between them is

$$d(X,Y) = \|X - Y\| = \left\| \begin{bmatrix} x_2 - x_1 \\ y_2 - y_1 \\ z_2 - z_1 \end{bmatrix} \right\|$$

$$= \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2 + (z_2 - z_1)^2} \qquad (P=2)$$

Distance between two points in n-dimension

If the points in n dimensional space are

$$X = \begin{bmatrix} x_1 \\ .. \\ x_n \end{bmatrix}$$

$$Y = \begin{bmatrix} y_1 \\ .. \\ y_n \end{bmatrix}$$

then the distance between them is

$$d(X,Y) = \|X - Y\| = \sqrt{\sum_{i=1}^{n}(x_i - y_i)^2}$$

Correlation in Statistics

Methods of correlation summarize the relationship between two variables in a single number called the correlation coefficient. The correlation coefficient is usually represented using the symbol r, and it ranges from -1 to +1.

A correlation coefficient quite close to 0, but either positive or negative, implies little or no relationship between the two variables. A correlation coefficient close to plus 1 means a positive relationship between the two variables, with increases in one of the variables being associated with increases in the other variable.

A correlation coefficient close to -1 indicates a negative relationship between two variables, with an increase in one of the variables being associated with a decrease in the other variable.

For example, there exists a correlation between two variables X and Y, which means the value of one variable is found to change in one direction, the value of the other variable is found to change either in the same direction (i.e. positive change) or in the opposite direction (i.e. negative change). Furthermore, if the correlation exists, it is linear, i.e. we can represent the relative movement of the two variables by drawing a straight line on graph paper.

Correlation Coefficient

The correlation coefficient, r, is a summary measure that describes the extent of the statistical relationship between two variables. The correlation coefficient is scaled so that it is always between -1 and +1. When r is close to 0 this means that there is little relationship between the variables and the farther away from 0 r is, in either the positive or negative direction, the greater the relationship between the two variables.

Types of Correlation

The scatter plot explains the correlation between the two attributes or variables. It represents how closely the two variables are connected. There can be three such situations to see the relation between the two variables –

- Positive Correlation – when the values of the two variables move in the same direction so that an increase/decrease in the value of one variable is followed by an increase/decrease in the value of the other variable.

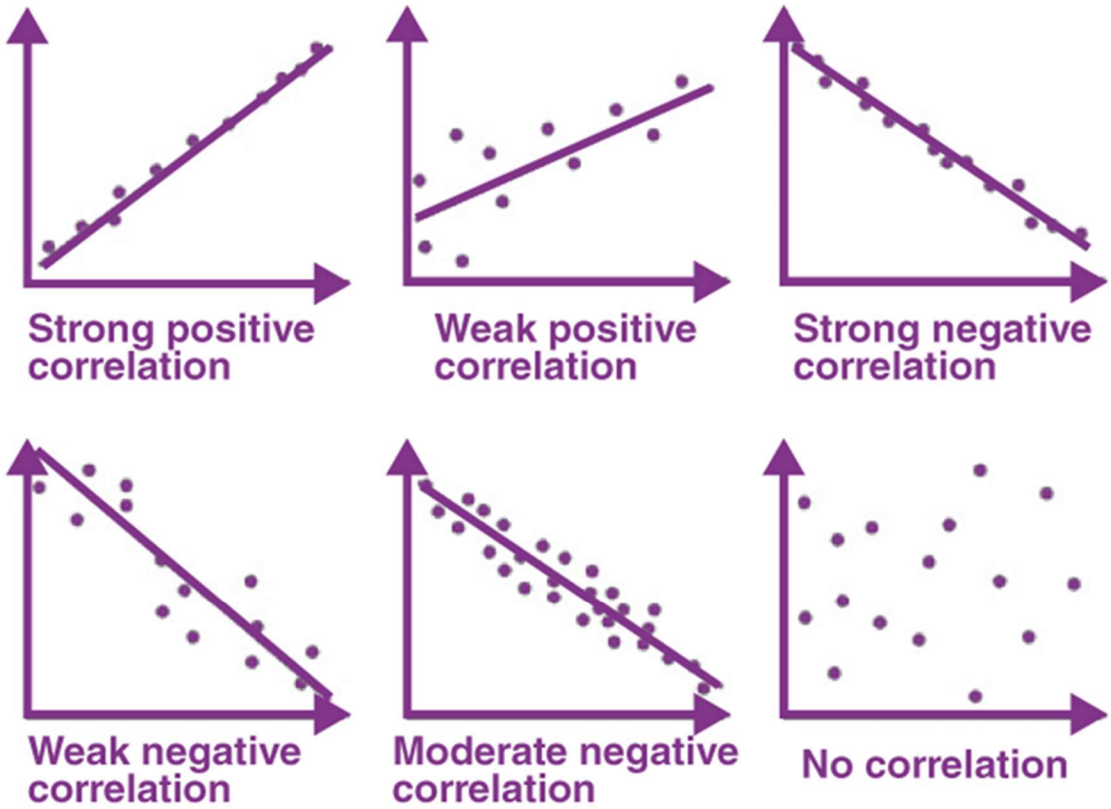
  For example-One example of positive correlation is the relationship between **employment and inflation**. High levels of employment require employers to offer higher salaries in order to attract new workers, and higher prices for their products in order to fund those

higher salaries. Conversely, periods of high unemployment experience falling consumer demand, resulting in downward pressure on prices and inflation.

- Negative Correlation – when the values of the two variables move in the opposite direction so that an increase/decrease in the value of one variable is followed by decrease/increase in the value of the other variable.

  For example-Examples of negative correlation are common in the investment world. A well-known example is the negative correlation between crude oil prices and airline stock prices. Jet fuel, which is derived from crude oil, is a large cost input for airlines and has a significant impact on their profitability and earnings.
  If the price of crude oil spikes up, it could have a negative impact on airlines' earnings and hence on the price of their stocks. But if the price of crude oil trends lower, this should boost airline profits and therefore their stock prices.

- No Correlation – when there is no linear dependence or no relation between the two variables.



Correlation Formula

Correlation shows the relation between two variables. Correlation coefficient shows the measure of correlation. To compare two datasets, we use the correlation formulas.

Pearson Correlation Coefficient Formula

The most common formula is the Pearson Correlation coefficient used for linear dependency between the data sets. The value of the coefficient lies between -1 to +1. When the coefficient comes down to zero, then the data is considered as not related. While, if we get the value of +1, then the data are positively correlated, and -1 has a negative correlation.

$$r = \frac{n(\sum xy) - (\sum x)(\sum y)}{\sqrt{[n\sum x^2 - (\sum x)^2][n\sum y^2 - (\sum y)^2]}}$$

Where n = Quantity of Information

Σx = Total of the First Variable Value

Σy = Total of the Second Variable Value

Σxy = Sum of the Product of first & Second Value

Σx^2 = Sum of the Squares of the First Value

Σy^2 = Sum of the Squares of the Second Value

Linear Correlation Coefficient Formula

The formula for the linear correlation coefficient is given by;

$$r_{xy} = \frac{n\sum_{i=1}^{n} x_i y_i - \sum_{i=1}^{n} x_i \sum_{i=1}^{n} y_i}{\sqrt{n\sum_{i=1}^{n} x_i^2 - (\sum_{i=1}^{n} x_i)^2}\sqrt{n\sum_{i=1}^{n} y_i^2 - (\sum_{i=1}^{n} y_i)^2}}$$

When using the Pearson correlation coefficient formula, you'll need to consider whether you're dealing with data from a sample or the whole population.The sample and population formulas differ in their symbols and inputs. A sample correlation coefficient is called *r*, while a population correlation coefficient is called rho, the Greek letter ρ.

Sample Correlation Coefficient Formula-

The formula is given by:

$$r_{xy} = \frac{S_{xy}}{S_x \; S_y} = \frac{\sum (xi - \bar{x})(yi - \bar{y})}{\sqrt{\left(\sum (xi - \bar{x})^2\right)\left(\sum (yi - \bar{y})^2\right.}}$$

Where, $S_x, S_y \rightarrow$ Sample Standard Deviation

$S_{xy} \rightarrow$ Sample Covariance

$\bar{X}, \bar{Y} \rightarrow$ SampleMean

Population Correlation Coefficient Formula

The population correlation coefficient between two random variables X,Y with expected values and and standard deviations and is defined as:

$$\rho_{X,Y} = \operatorname{corr}(X,Y) = \frac{\operatorname{cov}(X,Y)}{\sigma_X \sigma_Y} = \frac{\operatorname{E}[(X-\mu_X)(Y-\mu_Y)]}{\sigma_X \sigma_Y}, \quad \text{if } \sigma_X \sigma_Y > 0.$$

Where E is the expected value, cov is the covariance and corr is correlation coefficient.

Examples using Correlation Coefficient Formula

Example 1. Given the following population data. Find the Pearson correlation coefficient between x and y for this data. (Take 1√7 as 0.378)

x	600	800	1000
y	1200	1000	2000

Solution:

To simplify the calculation, we divide both x and y by 100.

x/100	**y/100**	**$x_i - \bar{x}$**	**$y_i - \bar{y}$**	**$(x_i - \bar{x})^2$**	**$(y_i - \bar{y})^2$**	**$(x_i - \bar{x})(y_i - \bar{y})$**
6	12	-2	-2	4	4	4
8	10	0	-4	0	16	0
10	20	2	6	4	36	12
$\bar{x}=8$	$\bar{y}=14$			$\Sigma(x_i-\bar{x})^2=8$	$\Sigma(y_i-\bar{y})^2=56$	$\Sigma(x_i-\bar{x})(y_i-\bar{y})=16$

Using the correlation coefficient formula,

Pearson correlation coefficient for population =

$$r = \frac{n(\sum xy) - (\sum x)(\sum y)}{\sqrt{[n\sum x^2 - (\sum x)^2][n\sum y^2 - (\sum y)^2]}}$$

r = 0.756

Answer: Pearson correlation coefficient = 0.756

Example 2. A survey was conducted in your city. Given is the following sample data containing a person's age and their corresponding income. Find out whether the increase in age has an effect on income using the correlation coefficient formula. (Use 1√181 as 0.074 and 1√2091 as 0.07)

Age	25	30	36	43
Income	30000	44000	52000	70000

Solution:
To simplify the calculation, we divide y by 1000.

Age (x_i)	**Income/1000 (y_i/1000)**	**$x_i - \bar{x}$**	**$y_i - \bar{y}$**	**$(x_i - \bar{x})^2$**	**$(y_i - \bar{y})^2$**	**$(x_i - \bar{x})(y_i - \bar{y})$**

25	30	-8.5	-19	72.25	361	161.5
30	44	-3.5	-5	12.25	25	17.5
36	52	2.5	3	6.25	9	7.5
43	70	9.5	21	90.25	441	199.5
$\bar{x}=33.5$	$\bar{y}=49$			$\Sigma(x_i-\bar{x})^2=181$	$\Sigma(y_i-\bar{y})^2=836$	$\Sigma(x_i-\bar{x})(y_i-\bar{y})=386$

Pearson correlation coefficient for sample

$$r = \frac{n(\sum xy) - (\sum x)(\sum y)}{\sqrt{[n\sum x^2 - (\sum x)^2][n\sum y^2 - (\sum y)^2]}}$$

Therefore r=0.9923

Answer: Yes, with the increase in age a person's income increases as well, since the Pearson correlation coefficient between age and income is very close to 1.

Example 3: Calculate the Correlation coefficient of given data

x	41	42	43	44	45
y	3.2	3.3	3.4	3.5	3.6

Solution:

Here n = 5

Let us find $\sum x$, $\sum y$, $\sum xy$, $\sum x^2$, $\sum y^2$

x	**y**	**xy**	**x^2**	**y^2**
41	3.2	131.2	1681	10.24
42	3.3	138.6	1764	10.89
43	3.4	146.2	1849	11.56
44	3.5	154	1936	12.25
45	3.6	162	2025	12.96
$\sum x = 215$	**$\sum y = 17$**	**$\sum xy = 732$**	**$\sum x^2 = 9255$**	**$\sum y^2 = 57.9$**

X values:

$\sum x = 215$

$\sum x^2 = 9255$

$\bar{x} = 43$

$\sum(x - \bar{x})^2 = 10$

Y values:

$\sum y = 17$

$\sum y^2 = 57.9$

$\sum(y - \bar{y})^2 = 0.1$

X and Y combined

N = 5

$\sum((x - \bar{x})(y - \bar{y})) = 1$

$\sum xy = 732$

R calculation:

$r = = 1/\sqrt{((10)(0.1))} = 1$

Since r = 1, this indicates a significant relation between x and y.

<u>Regression Analysis</u>

Regression analysis refers to assessing the relationship between the outcome variable and one or more variables. The outcome variable is known as the dependent and co-founders are known independent variables. The dependent variable is shown by "y" and independent variables are shown by "x" in regression analysis.

Linear Regression

Linear regression is a linear approach to modelling the relationship between the scalar components and one or more independent variables. If the regression has one independent variable, then it is known as a simple linear regression. If it has more than one independent variable, then it is known as multiple linear regression.

- A regression is a statistical technique that relates a dependent variable to one or more independent (explanatory) variables.

- A regression model is able to show whether changes observed in the dependent variable are associated with changes in one or more of the explanatory variables.
- It does this by essentially fitting a best-fit line and seeing how the data is dispersed around this line.
- Regression helps economists and financial analysts in things ranging from asset valuation to making predictions.
- In order for regression results to be properly interpreted, several assumptions about the data and the model itself must hold.

Formula for linear regression equation is given by:
y=a+bx
a and *b* are given by the following formulas:

$$a\,(intercept) = \frac{\sum y \sum x^2 - \sum x \sum xy}{(\sum x^2) - (\sum x)^2}$$

$$b\,(slope) = \frac{n \sum xy - (\sum x)(\sum y)}{n \sum x^2 - (\sum x)^2}$$

Where,
x and *y* are two variables on the regression line.
b = Slope of the line.
a = *y*-intercept of the line.
x = Values of the first data set.
y = Values of the second data set.

Solved Examples

Question: Find linear regression equation for the following two sets of data:

x	2	4	6	8
y	3	7	5	10

Solution:

x	y	x^2	xy
2	3	4	6
4	7	16	28
6	5	36	30
8	10	64	80
$\sum x = 20$	$\sum y = 25$	$\sum x^2 = 120$	$\sum xy = 144$

b= $\frac{n\sum xy-(\sum x)(\sum y)}{n\sum x^2-(\sum x)^2}$

b=0.95

a= $\frac{\sum y\sum x^2-\sum x\sum xy}{(\sum x^2)-(\sum x)^2}$

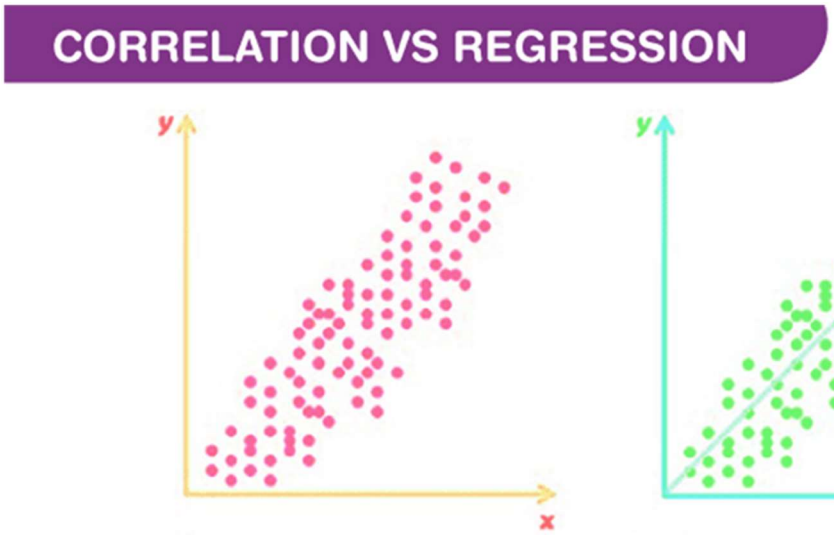
a = 1.5

Linear regression is given by:

y = a + bx

y = 1.5 + 0.95 x

Correlation and Regression Differences



There are some differences between Correlation and regression.

- Correlation shows the quantity of the degree to which two variables are associated. It does not fix a line through the data points. You compute a correlation that shows how much one variable changes when the other remains constant.
- Linear regression finds the best line that predicts y from x, but Correlation does not fit a line.
- Correlation is used when you measure both variables, while linear regression is mostly applied when x is a variable that is manipulated.

Comparison Between Correlation and Regression

Basis	**Correlation**	**Regression**
Meaning	A statistical measure that defines co-relationship or association of two variables.	Describes how an independent variable is associated with the dependent variable.

Dependent and Independent variables	No difference	Both variables are different.
Usage	To describe a linear relationship between two variables.	To fit the best line and estimate one variable based on another variable.
Objective	To find a value expressing the relationship between variables.	To estimate values of a random variable based on the values of a fixed variable.

References

- https://towardsdatascience.com
- https://www.geeksforgeeks.org
- https://www.javatpoint.com
- https://www.simplilearn.com

Introduction to Linear Algebra and statistics for AI :

- Basic matrix operations like matrix addition , subtraction , multiplication , transpose of matrix , identity matrix.---refer to any standard Maths text book of Class XI,XII.
- Brief introduction to vectors , unit vector , normal vector , Euclidean space.---refer to any standard Maths text book of Class XI,XII
- Probability distribution , frequency , mean , median and mode , variance and standard deviation , Gaussian distribution.------refer to any standard Maths text book of Class XI,XII

What is a Uniform Distribution?

The uniform distribution is a symmetric probability distribution where all outcomes have an equal likelihood of occurring. All values in the distribution have a constant probability, making them uniformly distributed. This distribution is also known as the rectangular distribution because of its shape in probability distribution plots.

Uniform Distribution Examples

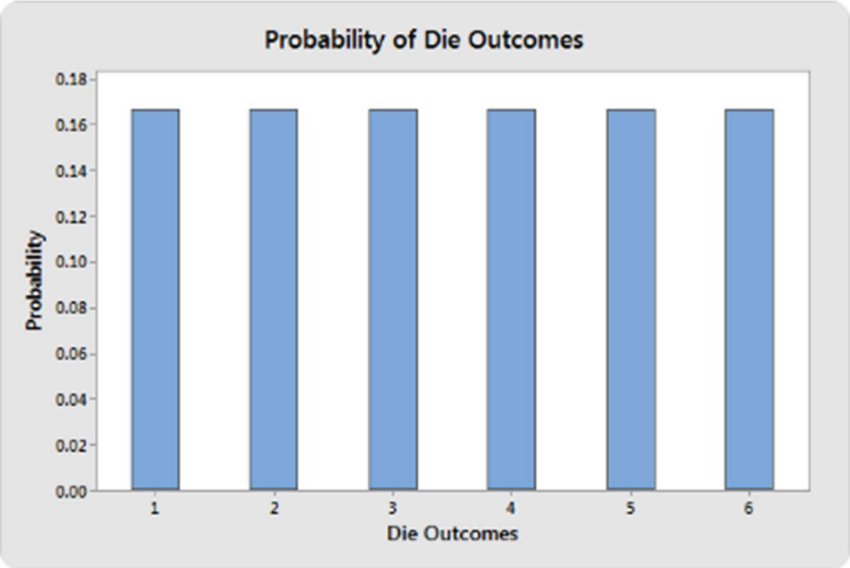
- Rolling dice and coin tosses.
- The probability of drawing any card from a deck of cards.
- Random sampling because that method depends on population members having equal chances.
- P-values in hypothesis tests follow the uniform distribution when the null hypothesis is true under certain conditions.
- Random number generators use the uniform distribution because no number should be more common than other numbers.
- Radioactive decay of particles over time

Analysts can use the uniform distribution to approximate new processes when there is insufficient data to estimate the actual distribution of outcomes.

Uniform distributions come in both discrete and continuous varieties of probability distributions.

Discrete: Each discrete value has an equal probability. For example, the chances of obtaining any of the six values on a die are equal.

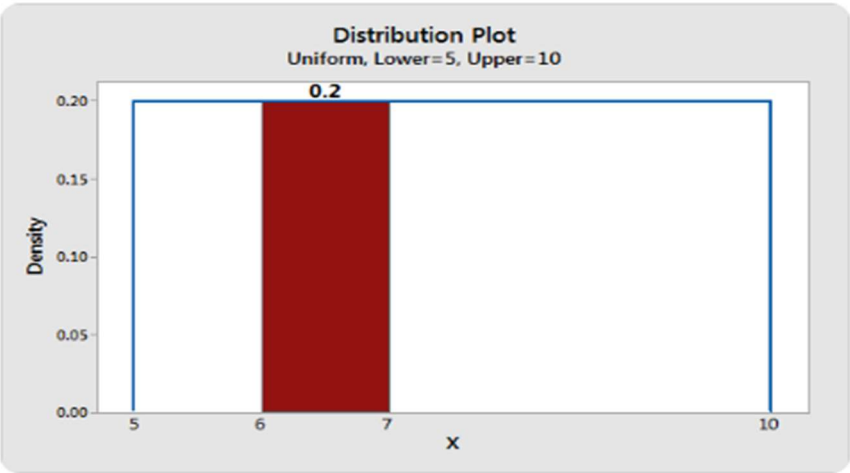
For discrete uniform distributions, finding the probability for each outcome is 1/n, where n is the number of outcomes. Rolling dice has six outcomes that are uniformly distributed. Therefore, each one has a likelihood of 1/6 = 0.167. The bar chart below displays the rectangular-shaped distribution.



Continuous: continuous data where all equal sized ranges have the same probability. For example, values are equally like to fall in the range of 0.1 – 0.2 as they have probability value within 0.4 – 0.5.

Both forms of the uniform distribution have two parameters, *a* and *b*. These values represent the smallest and largest values in the distribution.

In the example below, the distribution ranges from 5 to 10, which covers 5 units. The shaded area is one unit out of five or 1 / 5 = 20% of the total area. Hence, the probability for a value falling between 6 and 7 is 0.2. In fact, all one-unit ranges in this distribution have the same likelihood of 0.2.



The mean of the uniform distribution is (a + b) / 2.

What is a Random Variable?

A random variable is a variable where chance determines its value. They can take on either discrete or continuous values.

While randomness defines both discrete and continuous variables, their values are not entirely unpredictable. The probability of each value is well-defined and quantifiable using probability functions. By understanding the properties of these probability functions, we can make predictions and draw conclusions about real-world phenomena. These quantifiable properties make random variables a useful concept in statistics.

Discrete Random Variable

A discrete random variable has distinct values that are countable and finite or countably infinite. This data type often occurs when you are counting the number of event occurrences. For example, discrete random variables include the following:

- The number of heads that come up during a series of coin tosses.
- The number of library books checked out per hour.

Analysts denote the variable as X and its possible values as $x_1, x_2, \ldots, x_n$.

The probability of X having a value of x for its ith observation equals p_i: $P(X = x_i) = p_i$.

Using this notation, discrete random variables must satisfy these conditions:

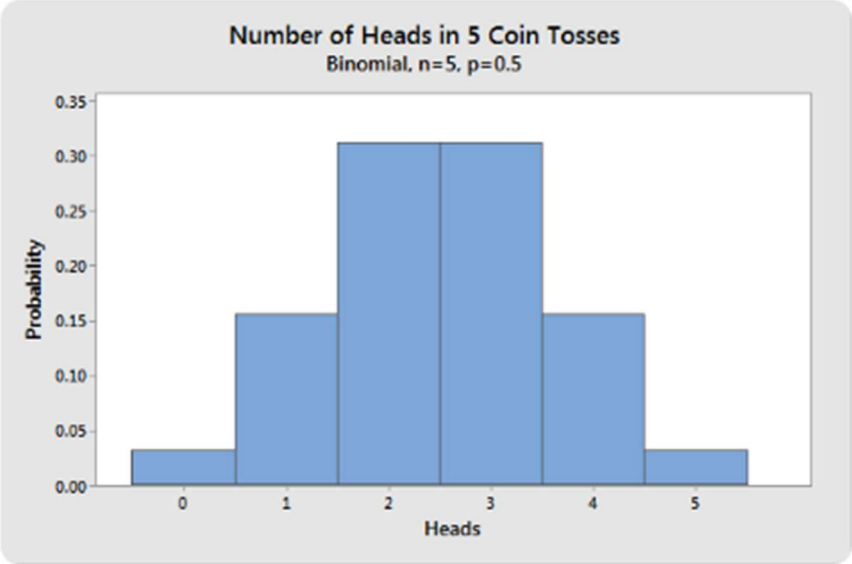
- All possible discrete values must have probabilities between zero and one:
  $0 < p_i \leq 1$.
- The total probability for all possible k values must equal 1:

  $p_1 + p_2 + p_3 + \ldots + p_k = 1$.

When these conditions are satisfied, one of the possible values will occur during every opportunity. The probability distribution of a discrete random variable is called a probability mass function (PMF).

Example of Discrete random sample

The number of heads that appear during a series of five coin tosses is a discrete random variable that follows the binomial distribution. We can use that distribution to determine the likelihood of obtaining 0 to 5 heads. The graph below displays the probability for each possible outcome.



Continuous Random Variable

A continuous random variable has values that are uncountably infinite and form a continuous range of values. They can take on any value within a range. In fact, there are infinite values between any two values.

This data type often occurs when we measure a quantity on a scale. For example, continuous random variables include the following:

- Height and weight.
- Time and duration.
- Temperatures.

Probabilities greater than zero only exist for ranges of values, such as $P(a \leq X \leq b)$, where a and b are the lower and upper bounds of the range.

A probability density function (PDF) describes the probability distribution of a continuous random variable. These functions use a curve displaying probability densities, which are ranges of one unit.

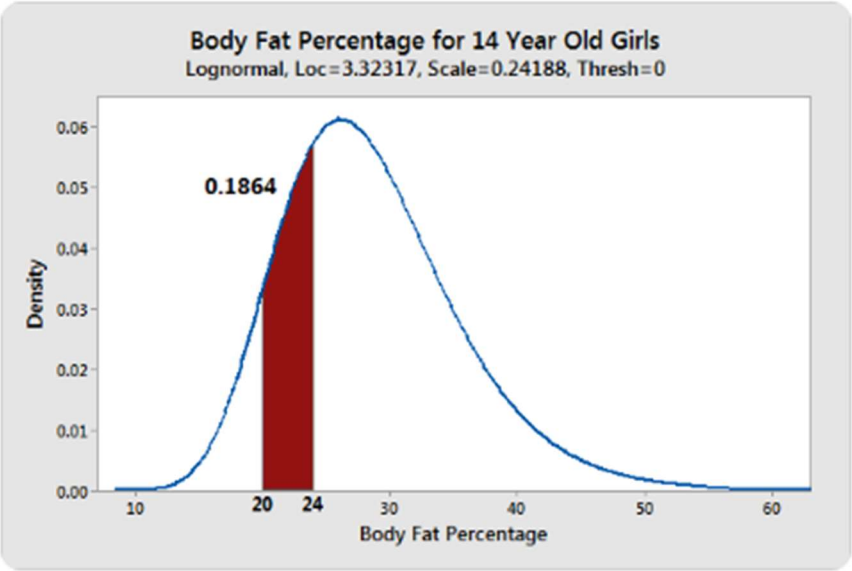
Continuous random variables must satisfy the following:

- Probabilities for all ranges of X are greater than or equal to zero: $P(a \leq X \leq b) \geq 0$.
- The total area under the curve equals one: $P(-\infty \leq X \leq +\infty) = 1$.

The likelihood of X falling within a particular range of values corresponds to the range's area under the PDF curve, which requires using an integral.

Example of continuous random sample

Body fat percentage is a continuous random variable. In preteen girls, it follows a lognormal distribution. Suppose a researcher needs to find participants with a body fat percentage between 20 and 24 percent. What is the likelihood that the next candidate will fall within that range?



The lognormal distribution graph indicates that the probability of body fat falling in the range of 20 to 24% is 0.1864. This information can help the researcher determine how many candidates they'll need to assess to obtain a sufficient sample size.

Hypothesis Testing

Hypothesis Testing is a tool to detect significant differences by comparing sample statistics with population parameters. It is used to infer the result of a hypothesis performed on sample data from a larger population.

Steps of Hypothesis Testing

1. State the Null hypothesis(H_0) and Alternate hypothesis (H_a)
2. Choose the level of significance($\propto$)
3. Determine the rejection area
4. Calculate test statistics and compare it with critical value

5. State the conclusion(reject the null hypothesis/failed to reject the null hypothesis)

Statistical Tests

Statistical Tests are conducted to test the hypothesis and to find the inferences about the population. For that samples are selected and various tests are performed on them to find the inference about the population under study.

Statistical tests are of two types:

1. Parametric Test
2. Non-Parametric Test

Parametric Tests

- Parametric tests are applied under the circumstances where the population is normally distributed or is assumed to be normally distributed.
- Parameters like mean, standard deviation etc are used.
- For example, T-test, Z-test, F-test, ANOVA, Pearson's Coefficient correlation.
- These are applied where the data is quantitative
- These are applied where the scale of measurement is either a interval or a ratio scale.

Non-Parametric Tests

- Non-Parametric tests are applied under the circumstances where the population is not normally distributed(skewed distribution) or is not assumed to be normally distributed.
- These texts are also called as Distribution free tests.
- Parameters like mean, standard deviation etc are not used.
- For example Chi-square test, U Test(Mann Whitney test) H-Test(Kruskal Wallis Test),Spearman's Rank Correlation Test.
- These are applied where the data is qualitative.
- These are applied where the scale of measurement is either an ordinal(ordered) or a nominal (name)scale.

Difference between Parametric Test and Non-Parametric Test

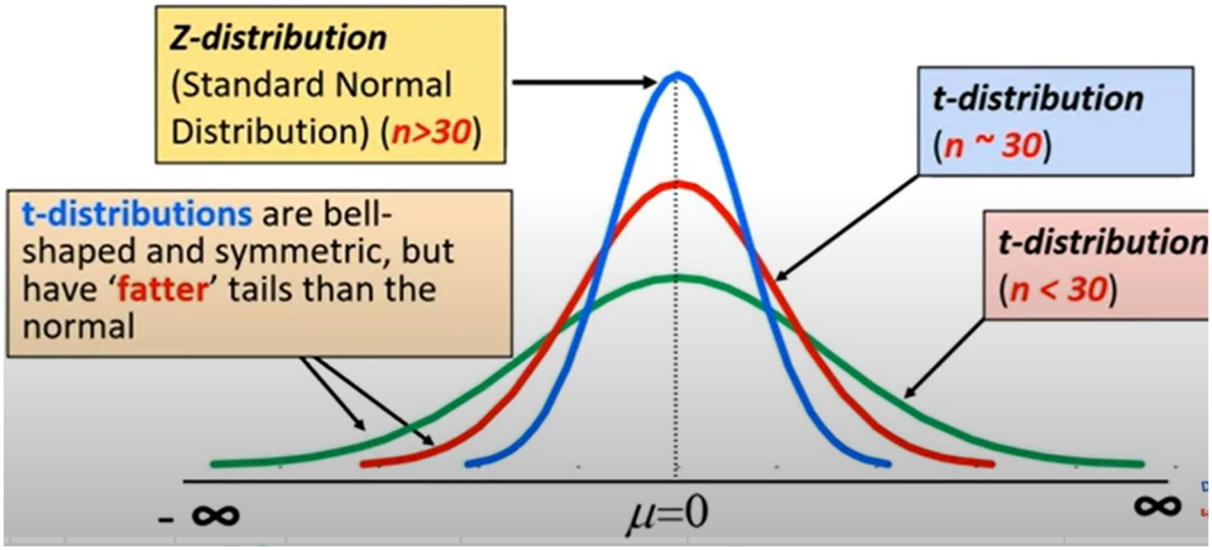
Parametric Test	Non-Parametric Test
1.Assumes the distribution to be normal	1.Does not assume the distribution to be normal
2.Make assumptions about he population	2.Does not make any assumptions about the population
3.Parameters such as mean, standard deviation etc. are used	3.No such Parameters are used
4. Applied in case of Quantitative data	4. Applied in case of qualitative data
5. Scale of measurement is either interval or ratio	5. . Scale of measurement is either ordinal or nominal
6. Uses mean as central tendency value	6. Uses median as central tendency value
7.More powerful as the possess the ability to reject the null Hypothesis, when it is false	7. Less powerful than parametric tests
8. Less robust	8. More robust as they are valid in a broader range of situations
9. For example, Z-test, T-test, ANOVA, F-test	9. For example, Chi-square test, U-test, H-test

Student's T-distribution hypothesis testing

Student's T-Distribution is a probability distribution that is used to calculate population parameters when the sample size is small and when the population varience (σ)is unknown.

Student's t-distribution was given by W.S.Gosset. he has published his studies under the name "Student". That's why it is called as Student's t-test.

Student's T-distribution is a continuous probability distribution that generalizes the standard normal distribution.



When to use T-distribution?

- Sample size<=30
- Population standard deviation(σ) is unknown
- Population distribution is unimodal

Properties of T-distribution

- Ranges from -∝ to +∝
- Bell shaped curve
- Student T distribution is different for different sample size
- Mean is zero
- Symetrical about mean

- Total area under T curve is equal to 1

Types of T-test

T test

One sample T test	**When Population mean is not equal to the sample mean**
Two sample paired T test	**Between two dependent groups at two different point of time**
Two sample independent T test	**Between two independent group**

Test Statistics for hypothesis testing of one sample T test

$$t=\frac{\bar{x}-\mu}{\frac{s}{\sqrt{n}}}$$

where,

$\bar{x}$ = sample mean

μ= Population mean

n= Sample size

s= Sample standard deviation

Acceptance Criteria

$t_{critical} <= t_{Test\ Statistic}$ -------- Reject Null Hypothesis

$t_{critical} > t_{Test\ Statistic}$ ---------- Failed to Reject NULL Hypothesis

The accommodation of the sample size is done through the concept of *degrees of freedom* (commonly abbreviated to *df*). The degrees of freedom represent the number of values in a statistical calculation that are free to vary. In the case of the *t*-distribution, the degrees of freedom are *N*-1 .

The significance level is the probability of rejecting the null hypothesis when it is true. For example, a significance level of 0.05 indicates a 5% risk of concluding that a difference exists when there is no actual difference. Lower significance levels indicate that you require stronger evidence before you will reject the null hypothesis.

Example:

https://www.youtube.com/watch?v=SgDlRSp-Olk

Test Statistics for hypothesis testing of two sample paired T test

$$t=\frac{\sum(x_1-x_2)/n}{\frac{s_d}{\sqrt{n}}}$$

where,

$x_1, x_2 =$	Two sample points
n=	Sample size
S_d=	Sample standard deviation

Test Statistics for hypothesis testing of two sample independentT test

$$t=\frac{(\bar{x}_1-\bar{x}_2)}{\sqrt{\frac{s_1^2}{n_1}+\frac{s_2^2}{n_2}}}$$

where,

$\overline{x_1} =$	first sample mean
$\overline{x_2} =$	second sample mean
n_1=	First sample size
n_2=	second sample size
S_1=	first Sample standard deviation
S_2=	Second Sample standard deviation

Z-Test

Z-test is the statistical test used to analyze whether two population means are different or not when the variances are known, and the sample size is large.

The z-test is based on the normal distribution.

The assumptions for Z-test are:

- All observations are independent.
- The size of the sample should be more than 30.
- The Z distribution is normal when the mean is 0, and the variance is 1.

The test statistic is defined by:

$$\text{Z-test}=\frac{\bar{x}-\mu}{\frac{\sigma}{\sqrt{n}}}$$

$\bar{x}$ is the sample mean
σ is the population standard deviation
n is the sample size
μ is the population mean

Example

Let's say that the mean score of students in a class is greater than 70 with a standard deviation of 10. If a sample of 50 students was selected with a mean score of 80, calculate the Z-value to check if there is enough evidence to support this claim at a 0.05 significance level.

Solution:

Here, the sample size is 50 and we know the standard deviation. This is a case of a right-tailed one-sample z test.

The Null hypothesis is the mean score is 70

The Alternative hypothesis is mean score is greater than 70

From the z-table, the critical value at alpha = 0.05 is 1.645
$\bar{x}$ = 80

$\mu = 70$
$n = 50$
$\sigma = 10$
Substituting the values in the formula, you will get the Z value to be equal to 7.09.

Since 7.09 > 1.645 thus, the null hypothesis is rejected and there is enough to support that the mean of the class is greater than 70.

Difference between T test and Z test

Z Test vs T Test

	Z Test	T Test
Assumption	Population standard deviation is known	Population standard deviation is unknown
Sample Size	Large sample size (n > 30)	Small sample size (n < 30)
Distribution	Z-distribution	T-distribution
Test Statistic	(Sample mean – Population mean) / (Population SD / √n)	(Sample mean – Population mean) / (Sample SD / √n)
Hypothesis Testing	Test for a population mean or proportion	Test for a population mean
Degrees of Freedom	Not applicable	n – 1
Application	Used when the population standard deviation is known and the sample size is large	Used when the population standard deviation is unknown or the sample size is small

Data Science Tools: Python vs R vs Excel

Python

Python is a very popular data science tool to support all four stages of the data lifecycle. Firstly, you can easily execute a collection of huge data. Even in an unstructured format, Python can help you to bring it in the right shape.

Secondly, data modelling is easy with Python. Proper modelling can always help you to observe patterns in the data. This helps business organizations to make proper decisions for the future.

Finally, Python gives way to clear data visualization. As a result, any business entity can make proper reports for the outcome. Furthermore, Python has numerous sub tools for each of the stages discussed above. Some of them are given below:

- Data Collection: In data collection, you can use the Python tools, such as Data APIs, Beautiful Soup, and Wget.
- Data Modelling: Spicy Ecosystem- NumPy, Imbalanced- learn, and Pandas, some data modelling tools that you can access while using Python.
- Data Visualization: With Python, you can use the tools like Matplotlib, MoviePy, and Seaborn. All these tools can provide you with the support for data visualization.
- NLP Tool: Python provides you with a bonus tool for string matching. It is known as FuzzyWuzzy, and it helps you to execute token ratios and comparison ratios. So, if you are looking to become a data scientist and an expert in Python, find a genuine online course in data science. Many of these courses are quite affordable and provide you with proper education.

R

Like Python, R is also a data science tool. It has been launched in the market by the R Foundation. Continuous development of R is active as the R Project is still ongoing. Some crucial facts about R are given here:-

R is mainly used for data analysis in the field of data science. As a result, you can handle the data or store it. Finally, you can analyze it too. As open-source software, it is quite adaptive. If you are new to data science as an expert, you can easily work with R. However, it depends on which industry you are working for and what type of data you are dealing with. The standard quality of R is also high as it is open-source software.

R is a convenient tool to use in statistical analysis. It can provide the analyzed report regarding the data graphically. Although other tools also allow this, the representation of R is simpler to understand.

The community of R is quite responsive. Many experts have in-depth knowledge about the computer language. In the community, 20 contributors work continuously fixing bugs and improving the tool according to the suggestions by the users.

Excel

Excel is a basic software most people learn while they are in school. However, it allows dealing with huge amounts of data. If you are a data scientist who deals with 2D data, Excel is the best tool for you.

As you learn Excel, you can edit and format the data easily. Moreover, documents in Excel can be easily shared. The Analysis ToolPak in Excel can be activated for accessing the advanced powers.

The Analysis ToolPak enables machine learning, and any data professional can easily carry out data analysis. A few data professionals might find Analysis ToolPak a bit outdated, but it can still be used in the present time.

Another fact why people find data analysis to be tricky is due to the type of coded functions it supports, especially with Excel. While working with data on Excel, you can only use functions like SAS and SPSS, which are quite tough. One good thing about Excel as a data science tool is that it enables users to use Python. However, you can only use it while accessing Excel on a Windows system.

Detailed Comparisons

Excel

When it comes to Excel as a data science tool, you should keep some usage scenarios, advantages, and disadvantages in mind.

Usage Scenarios

Excel can only support medium and small-scale business organizations as a data science tool. It is not suitable for large-scale business organizations as the volume of data remains quite high.

Simple data analysis can be executed on Excel. It can be the best data science tool for a school or bank. It allows the data scientist to execute regression analysis,

variance analysis, etc. So, it is clear that Excel can be used the best for dealing with the data generated from a general office.

Excel does not provide the user with a platform to create data analysis reports as a data science tool. While using Excel for data analysis, you have to use Word and PowerPoint to create reports. It is convenient for data visualization. You can easily make charts on it. So, concluding can be an easy task.

Advantages of Excel

Excel is a well-known software, and it has many advantages. The most significant ones are:-

- Easy Learning: All operations on Excel are quite easy to learn. A beginner who wants to be a successful data scientist can start learning Excel at first. In the data analytics course from XLRI, you can learn the ways to handle data on Excel.
- Enables Multiple Operations: Excel allows the users to do a lot of things with data. As mentioned earlier, data visualization is easy in Excel. Moreover, you can make dynamic charts on it too. Simple reports can be made on Excel, while the complicated ones need support.
- Learning the Basic Operations: You can learn all basic operations related to data science on Excel. This can ease your Python and R learning process.

Disadvantages of Excel

Despite the advantages of Excel mentioned above, there are some disadvantages too. Have a look:-

- Chances of Data Stuttering: You know that Excel can only fit medium and small organizations. This fact is quite right, as data stuttering is normal in its case. Excel might not be the right option when it comes to dealing with huge amounts of data.
- VBA is Necessary: To apply data science on a tool like Excel, VBA (Visual Basic Applications) is necessary. It is a tough programming language that might take a lot of time to learn. So, most data science professionals avoid using Excel as a tool.

R

The usage scenarios, advantages, and disadvantages of R as a tool of data science are discussed below. Take a look if you strive to have a basic idea about R as compared to Excel.

Usage Scenarios

R has complete coverage in any area where there is a necessity for data. The functions of R can help the user cover the areas of both general and academic data analysis.

While you use R, you can work on multiple aspects of data science. R mainly helps a user with the cleansing of data. Moreover, it allows web crawling and data visualization too. Report output can also be done with R. The 'R markdown' enables the user to get the data analysis report output.

Both statistical modelling and statistical hypothesis testing can be done easily with R. There are different types of algorithms that come under both of these. They are given below:

- t-test.
- Variance analysis.
- Chi-square test.
- Logistic regression.
- Linear regression.
- Neural network.
- Tree model.

Advantages of R

Here are the key advantages you can get while learning the usage of R. In a data analytics course from XLRI, you can study R in depth and enjoy the following advantages:

- Easy to Learn: The primary advantage of R is the ease it provides the user to learn the usage.
- Centralized Learning: Centralized learning can make a student know all the basics of R in a mere 10 to 12 classes. You can learn about structuring, importing, exporting, and visualizing data after completing the basic course.
- Quicker Approach to Solving Problems: R has several help files on the network. These files can help the user solve particular problems in no time.

Disadvantages of R

As a data science tool, R has only a few disadvantages. They are:

- Less Speed: In data analytics, R as a data science tool is quite slower than other options.

- Complicated language: To some data science professionals, R is quite a complicated language to deal with. However, it is simpler than VBA, which is necessary for working with Excel.

Python

Like R, Python has more or less identical usage scenarios. As you use Python, you can work on similar aspects of data as R. What outshines Python than R is that you can execute data mining. In this aspect, it has already taken the lead in comparison to R. Similar to R, Python also demands proper programming from the user. The advantage of Python is that it allows a professional to take the scientific computing approach. It is just a branch of this language.

Apart from data science, Python is largely used in web designing. Game developers also use this language to design modern game interfaces. Lastly, all work related to operations and maintenance can be done with Python.

What is R

R is a popular programming language used for statistical computing and graphical presentation.

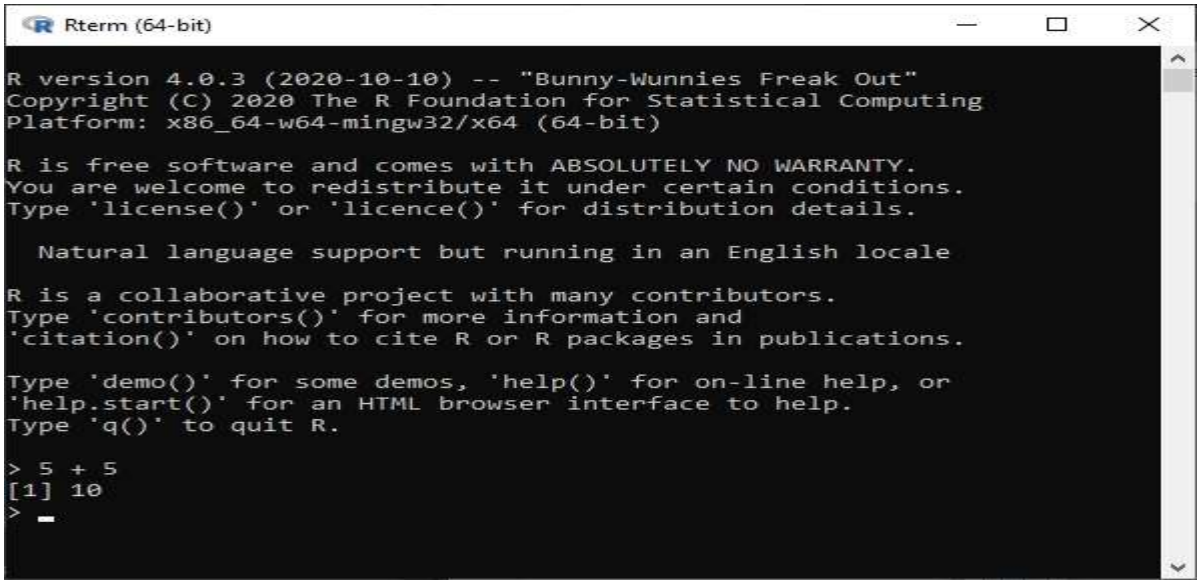
Its most common use is to analyze and visualize data.

Why Use R?

- It is a great resource for data analysis, data visualization, data science and machine learning
- It provides many statistical techniques (such as statistical tests, classification, clustering and data reduction)
- It is easy to draw graphs in R, like pie charts, histograms, box plot, scatter plot, etc++
- It works on different platforms (Windows, Mac, Linux)
- It is open-source and free
- It has a large community support
- It has many packages (libraries of functions) that can be used to solve different problems

How to Install R

To install R, go to https://cloud.r-project.org/ and download the latest version of R for Windows, Mac or Linux.
When you have downloaded and installed R, you can run R on your computer.
The screenshot below shows how it may look like when you run R on a Windows PC:



Syntax

To output text in R, use single or double quotes:

Example

```
"Hello World!"
```

Example

```
5 + 5
```

```
---- [1] 10
```

Print

Unlike many other programming languages, you can output code in R without using a print function:

Example

```
"Hello World!"
```

However, R does have a print() function available if you want to use it. This might be useful if you are familiar with other programming languages, such as Python, which often uses the print() function to output code.

Example

```
print("Hello World!")
```

And there are times you must use the print() function to output code, for example when working with for loops (which you will learn more about in a later chapter):

Example

```
for (x in 1:10) {
  print(x)
}
```

Conclusion: It is up to you whether you want to use the print() function to output code. However, when your code is inside an R expression (e.g. inside curly braces {} like in the example above), use the print() function to output the result.

Creating Variables in R

Variables are containers for storing data values.

R does not have a command for declaring a variable. A variable is created the moment you first assign a value to it. To assign a value to a variable, use the <- sign. To output (or print) the variable value, just type the variable name:

Example

```
name <- "John"
age <- 40

name   # output "John"
age    # output 40
```

From the example above, name and age are **variables**, while "John" and 40 are **values**.

In other programming language, it is common to use = as an assignment operator. In R, we can use both = and <- as assignment operators.

However, <- is preferred in most cases because the = operator can be forbidden in some context in R.

Print / Output Variables

Compared to many other programming languages, you do not have to use a function to print/output variables in R. You can just type the name of the variable:

Example

```
name <- "John Doe"

name # auto-print the value of the name variable
```

However, R does have a print() function available if you want to use it. This might be useful if you are familiar with other programming languages, such as Python, which often use a print() function to output variables.

Example

```
name <- "John Doe"

print(name) # print the value of the name variable
```

And there are times you must use the print() function to output code, for example when working with for loops (which you will learn more about in a later chapter):

Example

```
for (x in 1:10) {
  print(x)
}
```

Conclusion: It is up to your if you want to use the print() function or not to output code. However, when your code is inside an R expression (for example inside curly braces {} like in the example above), use the print() function if you want to output the result.

Concatenate Elements

You can also concatenate, or join, two or more elements, by using the paste() function.

To combine both text and a variable, R uses comma (,):

Example

```
text <- "awesome"

paste("R is", text)
```

You can also use , to add a variable to another variable:

Example

```
text1 <- "R is"
text2 <- "awesome"

paste(text1, text2)
```

For numbers, the + character works as a mathematical operator:

Example

```
num1 <- 5
num2 <- 10

num1 + num2
```

If you try to combine a string (text) and a number, R will give you an error:

Example

```
num <- 5
text <- "Some text"

num + text
```

Result:

```
Error in num + text : non-numeric argument to binary operator
```

Multiple Variables

R allows you to assign the same value to multiple variables in one line:

Example

```
# Assign the same value to multiple variables in one line
var1 <- var2 <- var3 <- "Orange"

# Print variable values
var1
var2
var3
```

Variable Names

A variable can have a short name (like x and y) or a more descriptive name (age, carname, total_volume). Rules for R variables are:

- A variable name must start with a letter and can be a combination of letters, digits, period(.)
  and underscore(_). If it starts with period(.), it cannot be followed by a digit.
- A variable name cannot start with a number or underscore (_)
- Variable names are case-sensitive (age, Age and AGE are three different variables)
- Reserved words cannot be used as variables (TRUE, FALSE, NULL, if...)

Data Types

In R, variables do not need to be declared with any particular type, and can even change type after they have been set:

Example

```
my_var <- 30 # my_var is type of numeric
my_var <- "Sally" # my_var is now of type character (aka string)
```

R has a variety of data types and object classes. You will learn much more about these as you continue to get to know R.

Basic Data Types

Basic data types in R can be divided into the following types:

- numeric - (10.5, 55, 787)
- integer - (1L, 55L, 100L, where the letter "L" declares this as an integer)
- complex - (9 + 3i, where "i" is the imaginary part)
- character (a.k.a. string) - ("k", "R is exciting", "FALSE", "11.5")
- logical (a.k.a. boolean) - (TRUE or FALSE)

We can use the class() function to check the data type of a variable:

Example

```
# numeric
x <- 10.5
class(x)

# integer
x <- 1000L
class(x)

# complex
x <- 9i + 3
class(x)

# character/string
x <- "R is exciting"
class(x)

# logical/boolean
x <- TRUE
class(x)
```

```
---- [1] "numeric"
[1] "integer"
[1] "complex"
[1] "character"
[1] "logical"
```

R Numbers

There are three number types in R:

- numeric
- integer
- complex

Variables of number types are created when you assign a value to them:

Example

```
x <- 10.5   # numeric
y <- 10L    # integer
z <- 1i     # complex
```

Numeric

A numeric data type is the most common type in R, and contains any number with or without a decimal, like: 10.5, 55, 787:

Example

```
x <- 10.5
y <- 55

# Print values of x and y
x
y

# Print the class name of x and y
class(x)
class(y)
```

Integer

Integers are numeric data without decimals. This is used when you are certain that you will never create a variable that should contain decimals. To create an integer variable, you must use the letter L after the integer value:

Example

```
x <- 1000L
y <- 55L
```

```
# Print values of x and y
x
y

# Print the class name of x and y
class(x)
class(y)
```

Complex

A complex number is written with an "i" as the imaginary part:

Example

```
x <- 3+5i
y <- 5i

# Print values of x and y
x
y

# Print the class name of x and y
class(x)
class(y)
```

Type Conversion

You can convert from one type to another with the following functions:

- as.numeric()
- as.integer()
- as.complex()

Example

```
x <- 1L # integer
y <- 2 # numeric

# convert from integer to numeric:
a <- as.numeric(x)

# convert from numeric to integer:
```

```
b <- as.integer(y)

# print values of x and y
x
y

# print the class name of a and b
class(a)
class(b)
```

```
    [1] 1
[1] 2
[1] "numeric"
[1] "integer"
```

Simple Math

In R, you can use **operators** to perform common mathematical operations on numbers.

The + operator is used to add together two values:

Example

```
10 + 5
```

And the - operator is used for subtraction:

Example

```
10 - 5
```

Built-in Math Functions

R also has many built-in math functions that allows you to perform mathematical tasks on numbers.

For example, the min() and max() functions can be used to find the lowest or highest number in a set:

Example

```
max(5, 10, 15)

min(5, 10, 15)
```

sqrt()

The sqrt() function returns the square root of a number:

Example

```
sqrt(16)
```

abs()

The abs() function returns the absolute (positive) value of a number:

Example

```
abs(-4.7)
```

ceiling() and floor()

The ceiling() function rounds a number upwards to its nearest integer, and the floor() function rounds a number downwards to its nearest integer, and returns the result:

Example

```
ceiling(1.4)

floor(1.4)
```

String Literals

Strings are used for storing text.

A string is surrounded by either single quotation marks, or double quotation marks:

"hello" is the same as 'hello':

Example

```
"hello"
'hello'
```

```
---- [1] "hello"
[1] "hello"
```

Assign a String to a Variable

Assigning a string to a variable is done with the variable followed by the <- operator and the string:

Example

```
str <- "Hello"
str # print the value of str
```

Multiline Strings

You can assign a multiline string to a variable like this:

Example

```
str <- "Lorem ipsum dolor sit amet,
consectetur adipiscing elit,
sed do eiusmod tempor incididunt
ut labore et dolore magna aliqua."

str # print the value of str
```

However, note that R will add a "**\n**" at the end of each line break. This is called an escape character, and the **n** character indicates a **new line**.

If you want the line breaks to be inserted at the same position as in the code, use the cat() function:

Example

```
str <- "Lorem ipsum dolor sit amet,
consectetur adipiscing elit,
sed do eiusmod tempor incididunt
ut labore et dolore magna aliqua."

cat(str)
```

```
---- Lorem ipsum dolor sit amet,
consectetur adipiscing elit,
sed do eiusmod tempor incididunt
ut labore et dolore magna aliqua.
```

String Length

There are many usesful string functions in R.

For example, to find the number of characters in a string, use the nchar() function:

Example

```
str <- "Hello World!"

nchar(str)
```

Check a String

Use the grepl() function to check if a character or a sequence of characters are present in a string:

Example

```
str <- "Hello World!"

grepl("H", str)
grepl("Hello", str)
grepl("X", str)
```

Combine Two Strings

Use the paste() function to merge/concatenate two strings:

Example

```
str1 <- "Hello"
str2 <- "World"

paste(str1, str2)
```

Booleans (Logical Values)

In programming, you often need to know if an expression is **true** or **false**.

You can evaluate any expression in R, and get one of two answers, TRUE or FALSE.

When you compare two values, the expression is evaluated and R returns the logical answer:

Example

```
10 > 9    # TRUE because 10 is greater than 9
10 == 9   # FALSE because 10 is not equal to 9
10 < 9    # FALSE because 10 is greater than 9
```

You can also compare two variables:

Example

```
a <- 10
b <- 9

a > b
```

You can also run a condition in an if statement, which you will learn much more about in the if..else chapter.

Example

```
a <- 200
b <- 33

if (b > a) {
  print ("b is greater than a")
} else {
  print("b is not greater than a")
}
```

Operators

Operators are used to perform operations on variables and values.

In the example below, we use the + operator to add together two values:

Example

```
10 + 5
```

R divides the operators in the following groups:

- Arithmetic operators
- Assignment operators
- Comparison operators
- Logical operators
- Miscellaneous operators

R Arithmetic Operators

Arithmetic operators are used with numeric values to perform common mathematical operations:

Operator	Name
+	Addition
-	Subtraction
*	Multiplication
/	Division
^	Exponent
%%	Modulus (Remainder from division)
%/%	Integer Division

R Assignment Operators

Assignment operators are used to assign values to variables:

Example

```
my_var <- 3

my_var <<- 3
```

```
3 -> my_var

3 ->> my_var

my_var # print my_var
```

Note: `<<-` is a global assigner. You will learn more about this in the [Global Variable chapter](#).

It is also possible to turn the direction of the assignment operator.

`x <- 3` is equal to `3 -> x`

R Comparison Operators

Comparison operators are used to compare two values:

Operator	Name
==	Equal
!=	Not equal
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

R Logical Operators

Logical operators are used to combine conditional statements:

Operator	Description
&	Element-wise Logical AND operator. It returns TRUE if both elements are TRUE
&&	Logical AND operator - Returns TRUE if both statements are TRUE
|	Elementwise- Logical OR operator. It returns TRUE if one of the statement is TRUE
||	Logical OR operator. It returns TRUE if one of the statement is TRUE.
!	Logical NOT - returns FALSE if statement is TRUE

R Miscellaneous Operators

Miscellaneous operators are used to manipulate data:

Operator	Description
:	Creates a series of numbers in a sequence

%in%	Find out if an element belongs to a vector
%*%	Matrix Multiplication

The if Statement

An "if statement" is written with the if keyword, and it is used to specify a block of code to be executed if a condition is TRUE:

Example

```
a <- 33
b <- 200

if (b > a) {
  print("b is greater than a")
}
```

In this example we use two variables, a and b, which are used as a part of the if statement to test whether b is greater than a. As a is 33, and b is 200, we know that 200 is greater than 33, and so we print to screen that "b is greater than a".

R uses curly brackets { } to define the scope in the code.

Else If

The else if keyword is R's way of saying "if the previous conditions were not true, then try this condition":

Example

```
a <- 33
b <- 33

if (b > a) {
  print("b is greater than a")
} else if (a == b) {
```

```
  print ("a and b are equal")
}
```

In this example a is equal to b, so the first condition is not true, but the else if condition is true, so we print to screen that "a and b are equal".

You can use as many else if statements as you want in R.

If Else

The else keyword catches anything which isn't caught by the preceding conditions:

Example

```
a <- 200
b <- 33

if (b > a) {
  print("b is greater than a")
} else if (a == b) {
  print("a and b are equal")
} else {
  print("a is greater than b")
}
```

In this example, a is greater than b, so the first condition is not true, also the else if condition is not true, so we go to the else condition and print to screen that "a is greater than b".

You can also use else without else if:

Example

```
a <- 200
b <- 33

if (b > a) {
  print("b is greater than a")
} else {
  print("b is not greater than a")
}
```

Nested If Statements

You can also have if statements inside if statements, this is called *nested* if statements.

Example

```
x <- 41

if (x > 10) {
  print("Above ten")
  if (x > 20) {
    print("and also above 20!")
  } else {
    print("but not above 20.")
  }
} else {
  print("below 10.")
}
```

```
---- [1] "Above ten"
[1] "and also above 20!"
```

AND

The & symbol (and) is a logical operator, and is used to combine conditional statements:

Example : Test if a is greater than b, AND if c is greater than a:

```
a <- 200
b <- 33
c <- 500

if (a > b & c > a) {
  print("Both conditions are true")
}
```

OR

The | symbol (or) is a logical operator, and is used to combine conditional statements:

Example : Test if a is greater than b, or if c is greater than a:

```
a <- 200
b <- 33
c <- 500

if (a > b | a > c) {
  print("At least one of the conditions is true")
}
```

Loops

Loops can execute a block of code as long as a specified condition is reached.

Loops are handy because they save time, reduce errors, and they make code more readable.

R has two loop commands:

- while loops
- for loops

R While Loops

With the while loop we can execute a set of statements as long as a condition is TRUE:

Example

Print i as long as i is less than 6:

```
i <- 1
while (i < 6) {
  print(i)
  i <- i + 1
}
```

In the example above, the loop will continue to produce numbers ranging from 1 to 5. The loop will stop at 6 because 6 < 6 is FALSE.

The while loop requires relevant variables to be ready, in this example we need to define an indexing variable, i, which we set to 1.

Note: remember to increment i, or else the loop will continue forever.

Break

With the break statement, we can stop the loop even if the while condition is TRUE:

Example

Exit the loop if i is equal to 4.

```
i <- 1
while (i < 6) {
  print(i)
  i <- i + 1
  if (i == 4) {
    break
  }
}
```

The loop will stop at 3 because we have chosen to finish the loop by using the break statement when i is equal to 4 (i == 4).

Next

With the next statement, we can skip an iteration without terminating the loop:

Example

Skip the value of 3:

```
i <- 0
while (i < 6) {
  i <- i + 1
  if (i == 3) {
    next
  }
  print(i)
}
```

When the loop passes the value 3, it will skip it and continue to loop.

To demonstrate a practical example, let us say we play a game of Yahtzee!

Example

Print "Yahtzee!" If the dice number is 6:

```
dice <- 1
while (dice <= 6) {
  if (dice < 6) {
    print("No Yahtzee")
  } else {
    print("Yahtzee!")
  }
  dice <- dice + 1
}
```

If the loop passes the values ranging from 1 to 5, it prints "No Yahtzee". Whenever it passes the value 6, it prints "Yahtzee!".

For Loops

A for loop is used for iterating over a sequence:

Example

```
for (x in 1:10) {
  print(x)
}
```

This is less like the for keyword in other programming languages, and works more like an iterator method as found in other object-orientated programming languages.

With the for loop we can execute a set of statements, once for each item in a vector, array, list, etc..

Example : Print every item in a list:

```
fruits <- list("apple", "banana", "cherry")

for (x in fruits) {
  print(x)
}
```

Example : Print the number of dices:

```
dice <- c(1, 2, 3, 4, 5, 6)

for (x in dice) {
  print(x)
}
```

The for loop does not require an indexing variable to set beforehand, like with while loops.

Break

With the break statement, we can stop the loop before it has looped through all the items:

Example

Stop the loop at "cherry":

```
fruits <- list("apple", "banana", "cherry")

for (x in fruits) {
  if (x == "cherry") {
    break
  }
  print(x)
}
```

The loop will stop at "cherry" because we have chosen to finish the loop by using the break statement when x is equal to "cherry" (x == "cherry").

Next

With the next statement, we can skip an iteration without terminating the loop:

Example

Skip "banana":

```
fruits <- list("apple", "banana", "cherry")

for (x in fruits) {
  if (x == "banana") {
    next
```

```
  }
  print(x)
}
```

When the loop passes "banana", it will skip it and continue to loop.

To demonstrate a practical example, let us say we play a game of Yahtzee!

Example : **Print "Yahtzee!" If the dice number is 6:**

```
dice <- 1:6

for(x in dice) {
  if (x == 6) {
    print(paste("The dice number is", x, "Yahtzee!"))
  } else {
    print(paste("The dice number is", x, "Not Yahtzee"))
  }
}
```

If the loop reaches the values ranging from 1 to 5, it prints "No Yahtzee" and its number. When it reaches the value 6, it prints "Yahtzee!" and its number.

Nested Loops

It is also possible to place a loop inside another loop. This is called a **nested loop**:

Example : Print the adjective of each fruit in a list:

```
adj <- list("red", "big", "tasty")

fruits <- list("apple", "banana", "cherry")
  for (x in adj) {
    for (y in fruits) {
      print(paste(x, y))
  }
}
```

```
---- [1] "red apple"
[1] "red banana"
[1] "red cherry"
[1] "big apple"
[1] "big banana"
[1] "big cherry"
```

```
[1] "tasty apple"
[1] "tasty banana"
[1] "tasty cherry"
```

R Functions

A function is a block of code which only runs when it is called.

You can pass data, known as parameters, into a function.

A function can return data as a result.

Creating a Function

To create a function, use the function() keyword:

Example

```
my_function <- function() { # create a function with the name my_function
  print("Hello World!")
}
```

Call a Function

To call a function, use the function name followed by parenthesis, like **my_function()**:

Example

```
my_function <- function() {
  print("Hello World!")
}

my_function() # call the function named my_function
```

Arguments

Information can be passed into functions as arguments.

Arguments are specified after the function name, inside the parentheses. You can add as many arguments as you want, just separate them with a comma.

The following example has a function with one argument (fname). When the function is called, we pass along a first name, which is used inside the function to print the full name:

Example

```
my_function <- function(fname) {
  paste(fname, "Griffin")
}

my_function("Peter")
my_function("Lois")
my_function("Stewie")
```

From a function's perspective:

A parameter is the variable listed inside the parentheses in the function definition.

An argument is the value that is sent to the function when it is called.

Number of Arguments

By default, a function must be called with the correct number of arguments. Meaning that if your function expects 2 arguments, you have to call the function with 2 arguments, not more, and not less:

Example : This function expects 2 arguments, and gets 2 arguments:

```
my_function <- function(fname, lname) {
  paste(fname, lname)
}

my_function("Peter", "Griffin")
```

If you try to call the function with 1 or 3 arguments, you will get an error:

Example : This function expects 2 arguments, and gets 1 argument:

```
my_function <- function(fname, lname) {
  paste(fname, lname)
}

my_function("Peter")
```

Default Parameter Value

The following example shows how to use a default parameter value.

If we call the function without an argument, it uses the default value:

Example

```
my_function <- function(country = "Norway") {
  paste("I am from", country)
}

my_function("Sweden")
my_function("India")
my_function() # will get the default value, which is Norway
my_function("USA")
```

```
---- [1] "I am from Sweden"
[1] "I am from India"
[1] "I am from Norway"
[1] "I am from USA"
```

Return Values

To let a function return a result, use the return() function:

Example

```
my_function <- function(x) {
  return (5 * x)
}

print(my_function(3))
print(my_function(5))
print(my_function(9))
```

The output of the code above will be:

```
[1] 15
[1] 25
[1] 45
```

Data Structures in R Programming

A data structure is a particular way of organizing data in a computer so that it can be used effectively. The idea is to reduce the space and time complexities of

different tasks. Data structures in R programming are tools for holding multiple values.

R's base data structures are often organized by their dimensionality (1D, 2D, or nD) and whether they're homogeneous (all elements must be of the identical type) or heterogeneous (the elements are often of various types). This gives rise to the six data types which are most frequently utilized in data analysis.

The most essential data structures used in R include:

- **Vectors**
- **Lists**
- **Dataframes**
- **Matrices**
- **Arrays**
- **Factors**

Vectors

A vector is an ordered collection of basic data types of a given length. The only key thing here is all the elements of a vector must be of the identical data type e.g homogeneous data structures. Vectors are one-dimensional data structures.

Example:

```
# R program to illustrate Vector

# Vectors(ordered collection of same data type)
X = c(1, 3, 5, 7, 8)

# Printing those elements in console
print(X)
```

Output:
[1] 1 3 5 7 8

Lists

A list is a generic object consisting of an ordered collection of objects. Lists are heterogeneous data structures. These are also one-dimensional data structures. A list can be a list of vectors, list of matrices, a list of characters and a list of functions and so on.

Example:

```r
# R program to illustrate a List

# The first attributes is a numeric vector
# containing the employee IDs which is
# created using the 'c' command here
empId = c(1, 2, 3, 4)

# The second attribute is the employee name
# which is created using this line of code here
# which is the character vector
empName = c("Debi", "Sandeep", "Subham", "Shiba")

# The third attribute is the number of employees
# which is a single numeric variable.
numberOfEmp = 4

# We can combine all these three different
# data types into a list
# containing the details of employees
# which can be done using a list command
empList = list(empId, empName, numberOfEmp)

print(empList)
```

Output:

```
[[1]]
[1] 1 2 3 4

[[2]]
[1] "Debi"    "Sandeep" "Subham"  "Shiba"

[[3]]
[1] 4
```

Dataframes

Dataframes are generic data objects of R which are used to store the tabular data. Dataframes are the foremost popular data objects in R programming because we are comfortable in seeing the data within the tabular form. They are two-dimensional, heterogeneous data structures. These are lists of vectors of equal lengths.

Data frames have the following constraints placed upon them:

- A data-frame must have column names and every row should have a unique name.
- Each column must have the identical number of items.
- Each item in a single column must be of the same data type.
- Different columns may have different data types.

To create a data frame we use the data.frame() function.

Example:

```
# R program to illustrate dataframe

# A vector which is a character vector
Name = c("Amiya", "Raj", "Asish")

# A vector which is a character vector
Language = c("R", "Python", "Java")

# A vector which is a numeric vector
Age = c(22, 25, 45)

# To create dataframe use data.frame command
# and then pass each of the vectors
# we have created as arguments
# to the function data.frame()
df = data.frame(Name, Language, Age)

print(df)
```

Output:

```
  Name Language Age
1 Amiya        R  22
2  Raj   Python  25
3 Asish     Java  45
```

Matrices

A matrix is a rectangular arrangement of numbers in rows and columns. In a matrix, as we know rows are the ones that run horizontally and columns are the ones that run vertically. Matrices are two-dimensional, homogeneous data structures.

Now, let's see how to create a matrix in R. To create a matrix in R you need to use

the function called matrix. The arguments to this matrix() are the set of elements in the vector. You have to pass how many numbers of rows and how many numbers of columns you want to have in your matrix and this is the important point you have to remember that by default, matrices are in column-wise order.

Example:

```
# R program to illustrate a matrix

A = matrix(
  # Taking sequence of elements
  c(1, 2, 3, 4, 5, 6, 7, 8, 9),

  # No of rows and columns
  nrow = 3, ncol = 3,

  # By default matrices are
  # in column-wise order
  # So this parameter decides
  # how to arrange the matrix
  byrow = TRUE
)

print(A)
```

Output:

```
     [,1] [,2] [,3]
[1,]    1    2    3
[2,]    4    5    6
[3,]    7    8    9
```

Arrays

Arrays are the R data objects which store the data in more than two dimensions. Arrays are n-dimensional data structures. For example, if we create an array of dimensions (2, 3, 3) then it creates 3 rectangular matrices each with 2 rows and 3 columns. They are homogeneous data structures.

Now, let's see how to create arrays in R. To create an array in R you need to use the function called array(). The arguments to this array() are the set of elements in vectors and you have to pass a vector containing the dimensions of the array.

Example:

```
# R program to illustrate an array

A = array(
  # Taking sequence of elements
  c(1, 2, 3, 4, 5, 6, 7, 8),

  # Creating two rectangular matrices
  # each with two rows and two columns
  dim = c(2, 2, 2)
)

print(A)
```

Output:

```
, , 1

     [,1] [,2]
[1,]    1    3
[2,]    2    4

, , 2

     [,1] [,2]
[1,]    5    7
[2,]    6    8
```

Factors

Factors are the data objects which are used to categorize the data and store it as levels. They are useful for storing categorical data. They can store both strings and integers. They are useful to categorize unique values in columns like “TRUE” or “FALSE”, or “MALE” or “FEMALE”, etc.. They are useful in data analysis for statistical modeling.

Now, let’s see how to create factors in R. To create a factor in R you need to use the function called factor(). The argument to this factor() is the vector.

Example:

```
# R program to illustrate factors

# Creating factor using factor()
```

```r
fac = factor(c("Male", "Female", "Male",
               "Male", "Female", "Male", "Female"))

print(fac)
```

Output:

```
[1] Male   Female Male   Male   Female Male   Female
Levels: Female Male
```

Use link for further study on R

https://www.w3schools.com/r/default.asp

Use link for study on Rstudio

https://www.datacamp.com/tutorial/r-studio-tutorial